

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**TINY YOLOV3 CẢI TIẾN DỰA TRÊN KHỐI CBAM,
GSCONV, C3GHOST VÀ TUCKER DECOMPOSITION
CHO NHẬN DIỆN ĐỐI TƯỢNG THỜI GIAN THỰC
TRÊN THIẾT BỊ NHÚNG**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: PGS-TS. VÕ MINH HUÂN

SVTH: LÊ ĐÌNH THÁI SƠN

MSSV: 22119224

NGUYỄN MINH TRIẾT

MSSV: 22119242

TP. HỒ CHÍ MINH – 06/2026



TP. Hồ Chí Minh, ngày 26 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN

Họ và tên sinh viên: Lê Đình Thái Sơn- **MSSV:** 22119224

Nguyễn Minh Triết - **MSSV:** 22119242

Ngành: Công nghệ kỹ thuật máy tính

Lớp: 221191A-221191A

Giảng viên hướng dẫn: PGS-TS.Võ Minh Huân

Ngày nhận đề tài: 14/03/2026

Ngày nộp đề tài: 26/06/2026

1. Tên đề tài Tiny YOLOv3 cải tiến dựa trên khối CBAM, GSConv, C3Ghost Và Tucker Decomposition Cho Nhận Diện Đối Tượng Thời Gian Thực Trên Thiết Bị Nhúng.

2. Các số liệu, tài liệu ban đầu: Kiến thức cơ bản về các môn Học sâu (Deep Learning), Thị giác máy tính (Computer Vision) và Hệ thống nhúng / Trí tuệ nhân tạo nhúng (Edge AI),...

3. Nội dung thực hiện đề tài

Đề tài "Nâng cao hiệu năng nhận diện đối tượng dựa trên kiến trúc Tiny-YOLOv3 cải tiến với 3 Head, CBAM, GSConv và C3Ghost" được triển khai bằng Python và PyTorch. Mô hình được cải tiến bằng cách bổ sung 3 Head để tăng khả năng phát hiện đối tượng ở nhiều kích thước, kết hợp CBAM nhằm nâng cao khả năng tập trung vào đặc trưng quan trọng, đồng thời sử dụng GSConv và C3Ghost để giảm độ phức tạp tính toán. Hiệu quả của mô hình được đánh giá thông qua các chỉ số độ chính xác, số tham số, FLOPs và FPS.

TRƯỞNG BỘ MÔN

(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Tiny YOLOv3 cải tiến dựa trên khối CBAM, GSConv, C3Ghost Và Tucker Decomposition Cho Nhận Diện Đối Tượng Thời Gian Thực Trên Thiết Bị Nhúng.

Sinh viên: + Lê Đình Thái Sơn

MSSV: 22119224

+ Nguyễn Minh Triết

MSSV: 22119242

Hướng dẫn: PGS-TS. Võ Minh Huân

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Vấn đề được đặt rõ ràng, mục tiêu cụ thể theo ba nhóm (tăng độ chính xác, tối ưu kiến trúc, tăng tốc suy diễn). Đề tài có tính mới khi kết hợp CBAM, GSConv, C3Ghost và Tucker Decomposition trong một pipeline hoàn chỉnh cho bài toán Edge AI.

Có tính cấp thiết và ứng dụng thực tiễn trong giám sát giao thông; hướng tiếp cận sáng tạo khi triển khai và kiểm chứng thực tế trên Raspberry Pi 4 thương mại phổ thông.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp ba giai đoạn hệ thống, có cơ sở lý thuyết rõ ràng cho từng kỹ thuật. Thiết kế ablation study giúp định lượng đóng góp của từng thành phần một cách minh bạch.

Bộ chỉ số đánh giá (mAP@0.5, FPS, GFLOPs) phù hợp; quy trình fine-tuning hai pha sau nén được thiết kế hợp lý. Thể hiện năng lực làm chủ cả thiết kế kiến trúc lẫn tối ưu hóa nhúng.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Mô hình cuối đạt mAP@0.5 trên 80% ; triển khai trên Raspberry Pi 4 đạt 6,0 FPS sau lượng tử hóa INT8. Kết quả được kiểm chứng đầy đủ qua bảng ablation và phân tích per-class.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận bám sát mục tiêu, nêu rõ đóng góp và hạn chế một cách trung thực. Đề xuất hướng phát triển cụ thể (QAT, tự động rank selection, nền tảng Jetson/Coral) có tính khả thi cao.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Báo cáo có bố cục sáu chương rõ ràng, đầy đủ mục lục và danh mục; định dạng nhất quán theo mẫu. Còn một số chú thích hình/bảng chưa thống nhất vị trí và lỗi typo nhỏ cần chỉnh sửa.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Nhóm làm chủ chuỗi công cụ từ huấn luyện (TensorFlow/Keras) đến nén mô hình, chuyển đổi ONNX INT8 và triển khai phần cứng thực tế. Thể hiện khả năng tích hợp hệ thống tốt.

7. Tài liệu trích dẫn
Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có 17 tài liệu tham khảo bao phủ các kỹ thuật cốt lõi, phần lớn theo đúng chuẩn APA. Một số mục chưa có DOI hoặc URL cần bổ sung thêm.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Đề tài không trùng lặp với công trình đã công bố; cách kết hợp pipeline ba giai đoạn và kiểm chứng trên phần cứng nhúng thương mại là đóng góp độc lập.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa:

Đề tài còn các hạn chế: tốc độ 5–6 FPS chưa đạt chuẩn real-time ; dữ liệu thiếu các điều kiện thời tiết phức tạp ; hệ số nén Tucker được chọn thủ công ; và chưa đối chứng PTQ với QAT để làm rõ nguyên nhân giảm độ chính xác..

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Nhóm thể hiện tinh thần nghiêm túc, chủ động và nỗ lực tốt trong suốt quá trình thực hiện đề tài có phạm vi rộng và yêu cầu kỹ thuật cao.

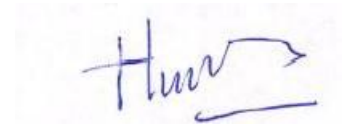
Đề nghị của giảng viên hướng dẫn

Ghi rõ: “Bảo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp

Tp. HCM, ngày 27 tháng 6. năm 2026

Người nhận xét



Võ Minh Huân

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện khóa luận xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Võ Minh Huân. Thầy đã luôn tận tình hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Những lời góp ý, chỉ bảo và đánh giá của thầy qua các buổi báo cáo đã giúp nhóm nhìn nhận ra những khuyết điểm, từ đó vượt qua các khó khăn về mặt kỹ thuật, nội dung cũng như hoàn thiện hình thức của bài báo cáo.

Tiếp theo, nhóm xin gửi lời tri ân đến quý thầy cô trong khoa Điện – Điện tử của Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh. Cảm ơn quý thầy cô đã truyền đạt những kiến thức quý báu từ cơ bản đến nâng cao, tạo tiền đề và nền tảng vững chắc cho việc thực hiện khóa luận này. Do kiến thức và kinh nghiệm thực tế còn hạn hẹp, nội dung của bài khóa luận chắc chắn khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét từ quý thầy cô để có thể tiếp thu và hoàn thiện đề tài hơn nữa.

Sau cùng, nhóm thực hiện xin kính chúc quý Thầy/Cô thật nhiều sức khỏe, luôn tràn đầy nhiệt huyết và gặt hái được nhiều thành công trong sự nghiệp giáo dục.

Nhóm thực hiện đề tài

Lê Đình Thái Sơn

Nguyễn Minh Triết

TÓM TẮT

Đề tài đề xuất và hiện thực một pipeline cải tiến ba giai đoạn cho mô hình phát hiện đối tượng Tiny YOLOv3, tập trung vào bài toán nhận diện người đi bộ và phương tiện giao thông trên thiết bị nhúng phổ thông.

Giai đoạn một bổ sung đầu phát hiện thứ ba kết hợp cơ chế chú ý CBAM để tăng khả năng nhận diện đối tượng nhỏ và đa tỉ lệ. Giai đoạn hai thay thế các khối tích chập trong phần neck và head bằng kiến trúc GSConv-v2 và C3Ghost-v2 nhẹ hơn, kết hợp cải tiến hàm mất mát và chiến lược huấn luyện, giúp nâng cao độ chính xác trong khi giảm đáng kể chi phí tính toán. Giai đoạn ba áp dụng Tucker Decomposition và lượng tử hóa INT8 qua ONNX Runtime để nén mô hình về kích thước phù hợp với bộ nhớ thiết bị nhúng.

Kết quả thực nghiệm cho thấy mô hình đề xuất cải thiện đáng kể độ chính xác so với kiến trúc gốc, đồng thời giảm mạnh số tham số và chi phí tính toán sau khi nén. Hệ thống được triển khai và kiểm chứng thực tế trên Raspberry Pi 4 không có bộ tăng tốc phần cứng chuyên dụng, đạt tốc độ suy diễn đáp ứng yêu cầu giám sát điểm cố định tốc độ thấp, khẳng định tính khả thi của phương pháp đề xuất trên phần cứng nhúng thương mại phổ thông.

MỤC LỤC

LỜI CẢM ƠN.....	i
TÓM TẮT.....	ii
DANH MỤC HÌNH ẢNH.....	viii
DANH MỤC BẢNG	ix
CÁC TỪ VIẾT TẮT	x
CHƯƠNG 1: GIỚI THIỆU	1
1.1 TỔNG QUAN.....	1
1.2 MỤC TIÊU	2
1.3 BỐ CỤC ĐỒ ÁN.....	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	6
2.1 TỔNG QUAN VỀ OBJECT DETECTION	6
2.1.1 Định nghĩa và phân loại.....	6
2.1.2 Anchor box	6
2.1.3 Chỉ số IoU và CIoU Loss	7
2.1.4 Phát hiện đa tỉ lệ	8
2.1.5 Đánh giá mô hình – mAP@0.5	9
2.2 KIẾN TRÚC TINY YOLOV3.....	9
2.2.1 Tổng quan về Tiny YOLOv3	9
2.2.2 Khởi tích chập cơ bản (DarknetConv2D_BN_Leaky)	9
2.2.3 Kiến trúc trích xuất đặc trưng (Backbone).....	10
2.2.4 Cổ mạng (Neck) và Hai nhánh phát hiện (Detection Heads).....	11
2.3 CƠ CHẾ ATTENTION – CBAM.....	12
2.3.1 Tổng quan về CBAM	12
2.3.2 Module Channel Attention (Chú ý theo Kênh)	12

2.3.3 Module Spatial Attention (Chú ý theo Không gian).....	13
2.3.4 Thực nghiệm tính hiệu quả của CBAM.....	14
2.4 GROUPED SHUFFLE CONVOLUTION.....	15
2.4.1 Hạn chế của tích chập tiêu chuẩn và động lực đề xuất.....	15
2.4.2 Kiến trúc và nguyên lý hoạt động của GSConv	16
2.4.3 Cơ chế Channel Shuffle.....	17
2.4.4 Hiệu quả cắt giảm tính toán của phương pháp	18
2.4.5 Thực nghiệm trên số đề so sánh GSConv và Standard Conv	18
2.5 CROSS STAGE PARTIAL, GHOST-BOTTLENECK VÀ C3GHOST	19
2.5.1 Tổng quan về sự dư thừa trong bản đồ đặc trưng.....	19
2.5.2 Tích chập Ghost (GhostConv)	20
2.5.3 Khối cổ chai Ghost (GhostBottleneck).....	21
2.5.4 Kiến trúc phân nhánh Cross Stage Partial (CSP)	22
2.5.5 Khối tích hợp C3Ghost.....	22
2.6 NÉN MÔ HÌNH BẰNG PHƯƠNG PHÁP TUCKER DECOMPOSITION	23
2.6.1 Nhu cầu nén mô hình cho thiết bị nhúng.....	23
2.6.2 Phân tích Tensor trọng số Conv2D.....	23
2.6.3 Tái cấu trúc thành ba lớp tích chập nhỏ.....	24
2.6.4 Xác định Rank và hệ số nén	24
CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT	26
3.1 TỔNG QUAN PIPELINE CẢI TIẾN	26
3.1.1 Chiến lược cải tiến tổng thể.....	26
3.1.2 Sơ đồ luồng xử lý ba giai đoạn.....	26
3.1.3 Môi liên hệ logic giữa các giai đoạn	27
3.2 GIAI ĐOẠN 1 – THÊM DETECTION HEAD THỨ 3 + CBAM.....	28

3.2.1	Bổ sung đầu phát hiện thứ ba với tỷ lệ 52×52	28
3.2.2	Tích hợp cơ chế chú ý CBAM.....	29
3.2.3	Kiến trúc tổng thể sau giai đoạn 1	30
3.3	GIAI ĐOẠN 2 – GSConv + C3GHOST V2 TRONG NECK VÀ HEAD	32
3.3.1	Động lực cải tiến.....	32
3.3.2	Tích hợp C3Ghost trong phần Neck.....	32
3.3.3	Tích hợp GSConvV2 tại Head 1	32
3.3.4	Kiến trúc tổng thể mạng nơ-ron sau Cải tiến 2	32
3.4	GIAI ĐOẠN 3 - NÉN MODEL BẰNG PHƯƠNG PHÁP TUCKER DECOMPOSITION.....	34
3.4.1	Lựa chọn lớp mục tiêu tối ưu	34
3.4.2	Thiết lập hệ số nén (Compression Factor) cho từng lớp	34
3.4.3	Tái cấu trúc lớp Conv2D thành khối ba lớp nối tiếp	35
3.4.4	Quy trình tinh chỉnh (Fine-tuning) hai pha sau nén	36
3.4.5	Tổng hợp kết quả và nhận xét.....	37
CHƯƠNG 4:	THỰC NGHIỆM VÀ KẾT QUẢ.....	38
4.1	DATASET.....	38
4.1.1	Nguồn dữ liệu	38
4.1.2	Thống kê tập dữ liệu	38
4.1.3	Phân tích sự mất cân bằng dữ liệu.....	39
4.1.4	Cân bằng dữ liệu.....	39
4.2	THIẾT LẬP HUẤN LUYỆN	41
4.2.1	Cấu hình phần cứng và phần mềm	41
4.2.2	Thiết lập huấn luyện	42
4.2.3	Thiết lập nén mô hình Tucker.....	44

4.3 KẾT QUẢ THỰC NGHIỆM.....	46
4.3.1 Kết quả đánh giá độ chính xác	46
4.3.2 Độ phức tạp và tốc độ suy luận	48
4.3.3 Kết quả sau nén Tucker	49
4.4 SO SÁNH VỚI CÁC MODEL.....	50
4.4.1 Thiết kế thực nghiệm ablation	50
4.4.2 Phân tích từng bước cải tiến	52
4.4.3 Đánh giá trade-off accuracy và tốc độ.....	53
4.4.4 Phân tích theo lớp đối tượng.....	53
4.4.5. Kiểm thử mô hình dự đoán với tập test	54
4.5 SO SÁNH CÁC MÔ HÌNH SAU KHI NÉN	55
4.5.1 Phương pháp Tucker Decomposition	55
4.5.2 Phân tích kết quả.....	56
4.5.3 Kết luận.....	57
4.6 PHÂN TÍCH KẾT QUẢ.....	57
CHƯƠNG 5: TRIỂN KHAI TRÊN THIẾT BỊ NHÚNG.....	59
5.1 MÔI TRƯỜNG TRIỂN KHAI.....	59
5.1.1 Lựa chọn nền tảng phần cứng.....	59
5.1.2 Môi trường phần mềm	60
5.1.3 Ràng buộc tài nguyên và yêu cầu thiết kế	61
5.2 QUY TRÌNH DEPLOY	61
5.2.1 Tổng quan quy trình triển khai	61
5.2.2 Chuyển đổi sang ONNX và Lượng tử hóa INT8 tĩnh	62
5.2.3 Thiết lập môi trường thực thi trên thiết bị nhúng	63
5.3 KẾT QUẢ TRÊN BOARD	63

5.3.1. Điều kiện đo lường	63
5.3.2 Kết quả tốc độ suy diễn trên video thực tế	64
5.4 NHẬN XÉT	68
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	70
6.1 KẾT LUẬN	70
6.2 HẠN CHẾ.....	71
6.3 HƯỚNG PHÁT TRIỂN	71
TÀI LIỆU THAM KHẢO	73

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ kiến trúc luồng dữ liệu của mạng Tiny YOLOv3 nguyên bản.....	10
Hình 2.2: Kiến trúc tổng thể của module CBAM (Channel Attention Module và Spatial Attention Module)	12
Hình 2.3: Kiến trúc tổng thể của khối GSConv (Grouped Shuffle Convolution)	16
Hình 3.1: Pipeline cải tiến của đề tài.....	27
Hình 3.2: Sơ đồ cấu trúc nhánh phát hiện thứ ba (Head 3) tại độ phân giải 52x52	29
Hình 3.3: Sơ đồ kiến trúc tổng thể sau giai đoạn 1	31
Hình 3.4: Kiến trúc tổng thể tích hợp C3GhostV2 và GSConvV2 tại Giai đoạn 2	33
Hình 3.5: Sự biến đổi kiến trúc của một lớp Conv2D qua Phân rã Tucker.....	36
Hình 3.6: Lưu đồ quy trình phân rã và tinh chỉnh trọng số mạng nơ-ron	37
Hình 4.1: Biểu đồ phân bố tập dữ liệu.....	40
Hình 4.2: So sánh hai nguồn dữ liệu	41
Hình 4.3: Kết quả sau khi test bằng CPU	46
Hình 4.4: Kết quả sau khi test bằng GPU.....	47
Hình 4.5: Kết quả sau khi test bằng GPU của mô hình sau nén.....	49
Hình 4.6: Kết quả GT trên tập test.....	54
Hình 4.7: kết quả dự đoán của mô hình.....	55
Hình 5.1: Lưu đồ tổng quan quy trình chuyển đổi và triển khai mô hình lên Raspberry Pi.	62
Hình 5.2. Kết quả đo benchmark trực tiếp các mô hình trên Raspberry Pi 4 (terminal output).....	64
Hình 5.3. Terminal output so sánh 8 mô hình trên Raspberry Pi 4: bảng tổng hợp inference time, FPS và log per-frame từ 4.109 khung hình kiểm thử.....	65

DANH MỤC BẢNG

Bảng 3.1: Tóm tắt mục tiêu và thành phần tác động của ba giai đoạn cải tiến	28
Bảng 3.2: Thiết lập hệ số nén Tucker Decomposition cho các lớp Conv2D.....	35
Bảng 4.1: Trình bày thống kê chi tiết số lượng ảnh và bounding box theo từng tập.	39
Bảng 4.2: So sánh phân bố dữ liệu trước và sau khi cân bằng.	39
Bảng 4.3: Các thư viện phần mềm chính.	42
Bảng 4.4: Thiết lập các thông số train	43
Bảng 4.5: Bảng các hệ số nén của ba config khác nhau.....	44
Bảng 4.6: Kết quả của 3 config nén khác nhau	44
Bảng 4.7: Tỷ lệ nén Tucker theo từng lớp	45
Bảng 4.8: Siêu tham số fine-tune sau nén Tucker	45
Bảng 4.9: Kết quả đánh giá mô hình đề xuất trên tập validation	46
Bảng 4.10: Thống kê TP / FP / FN trên tập validation	47
Bảng 4.11: Thông số mô hình và tốc độ suy luận	48
Bảng 4.12: So sánh mô hình Full-rank và Tucker Compressed	49
Bảng 4.13: So sánh các phiên bản mô hình (GPU GTX 1650, val 4.052 ảnh)	51
Bảng 4.14: So sánh các phiên bản mô hình (GPU GTX 1650, test 3138 ảnh)	52
Bảng 4.15: So sánh trước và sau Tucker Decomposition trên 3 kiến trúc (GPU GTX 1650, input 416×416).	56
Bảng 5.1: Thông số kỹ thuật của Raspberry Pi 4 Model B dùng trong triển khai	60
Bảng 5.2: So sánh tốc độ suy diễn và độ chính xác của ba mô hình trên Raspberry Pi 4.	65
Bảng 5.3: So sánh các mô hình trước nén và sau nén bằng 108 hình ảnh	67

CÁC TỪ VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ	Nghĩa tiếng việt
1	AP	Average Precision	Độ chính xác trung bình
2	ARM	Advanced RISC Machine	Kiến trúc vi xử lý nhúng phổ thông
3	BN	Batch Normalization	Chuẩn hóa theo batch trong quá trình huấn luyện
4	CBAM	Convolutional Block Attention Module	Module chú ý khối tích chập (kênh + không gian)
5	CIoU	Complete Intersection over Union	Chỉ số giao/hợp hoàn chỉnh, tính thêm khoảng cách tâm và tỉ lệ khung hình
6	CPU	Central Processing Unit	Đơn vị xử lý trung tâm
7	CSP	Cross Stage Partial	Kiến trúc mạng kết nối chéo giai đoạn một phần
8	FLOPs	Floating Point Operations	Số phép toán dấu phẩy động
9	FN	False Negative	Âm tính giả – đối tượng thực tế bị bỏ sót
10	FP	False Positive	Dương tính giả – phát hiện nhầm đối tượng không tồn tại
11	FPS	Frames Per Second	Số khung hình xử lý được mỗi giây
12	GFLOPs	Giga Floating Point Operations	Tỉ phép toán dấu phẩy động, đơn vị đo độ phức tạp tính toán
13	GSConv	Grouped Shuffle Convolution	Tích chập nhóm kết hợp trộn kênh (channel shuffle)
14	IoU	Intersection over Union	Chỉ số đo mức độ chồng lấp giữa hai bounding box
15	MSE	Mean Squared Error	Sai số bình phương trung bình
16	NMS	Non-Maximum Suppression	Thuật toán triệt tiêu phi cực đại để lọc bounding box trùng lặp
17	ReLU	Rectified Linear Unit	Hàm kích hoạt tuyến tính chỉnh lưu

18	YOLO	You Only Look Once	Kiến trúc phát hiện đối tượng thời gian thực – chỉ cần một lần duyệt ảnh
19	BBox	Bounding Box	Khung bao đối tượng – vùng hình chữ nhật xác định vị trí và kích thước của đối tượng trong ảnh

CHƯƠNG 1: GIỚI THIỆU

1.1 TỔNG QUAN

Trong những năm gần đây, sự phát triển mạnh mẽ của trí tuệ nhân tạo (AI) và thị giác máy tính đã mở ra nhiều ứng dụng thực tiễn trong đời sống, đặc biệt là bài toán phát hiện đối tượng trong thời gian thực. Trong lĩnh vực giao thông và an toàn đường bộ, việc tự động nhận diện phương tiện và người đi bộ đóng vai trò then chốt trong các hệ thống hỗ trợ lái xe tiên tiến (ADAS), giám sát giao thông thông minh và xe tự hành. Những hệ thống này đòi hỏi khả năng xử lý hình ảnh liên tục với độ trễ thấp, đồng thời phải hoạt động ổn định trong các điều kiện môi trường phức tạp như ánh sáng yếu, thời tiết xấu, hoặc khi đối tượng bị che khuất một phần.

Mặc dù vậy, một trong những thách thức lớn nhất của bài toán phát hiện đối tượng là sự đánh đổi giữa độ chính xác và tốc độ xử lý, đặc biệt khi triển khai trên các thiết bị nhúng (Edge AI) có tài nguyên tính toán hạn chế. Các mô hình học sâu có độ chính xác cao như YOLOv8, Faster R-CNN [3] hay DETR thường yêu cầu phần cứng GPU mạnh mẽ và tiêu thụ nhiều năng lượng, khiến chúng khó có thể triển khai trực tiếp trên các nền tảng nhúng giá rẻ. Ngược lại, các mô hình nhỏ gọn như Tiny YOLOv3 hay MobileNet-SSD tuy đáp ứng được yêu cầu về tốc độ nhưng lại suy giảm đáng kể độ chính xác, nhất là đối với các đối tượng nhỏ hoặc bị che khuất.

Tiny YOLOv3 là một kiến trúc tiêu biểu được thiết kế cho bài toán phát hiện thời gian thực nhờ số lượng tham số nhỏ (khoảng 8,7 triệu tham số) và tốc độ xử lý nhanh [2]. Tuy nhiên, khi áp dụng vào thực tế, mô hình này bộc lộ ba điểm hạn chế cốt lõi:

- Thiếu sự nhạy bén với đối tượng nhỏ: Mô hình chỉ sử dụng hai đầu phát hiện tại hai tỉ lệ đặc trưng (13×13 và 26×26), dẫn đến việc dễ bỏ sót người đi bộ ở khoảng cách xa hoặc xe máy trong đám đông.
- Thiếu cơ chế chú ý (Attention): Khối trích xuất đặc trưng (backbone) xử lý đồng đều toàn bộ bức ảnh mà không phân biệt được vùng chứa mục tiêu và vùng nền nhiễu, làm giảm độ chính xác trong các môi trường phức tạp.
- Khả năng tổng hợp đặc trưng yếu: Khối cổ (neck) kết nối backbone với các đầu phát hiện còn khá đơn giản, chưa tối ưu được khả năng tổng hợp thông tin đa tỉ lệ.

Bên cạnh đó, việc đưa các mô hình này lên phần cứng nhúng phổ thông như Raspberry Pi 4 (trang bị vi xử lý ARM Cortex-A72 bốn nhân, RAM 8GB, không có GPU hay bộ tăng tốc AI chuyên dụng) đặt ra bài toán khắt khe về tài nguyên. Để đạt được tốc độ suy diễn (inference) chấp nhận được, mô hình buộc phải được tối ưu hóa sâu cả về số lượng tham số lẫn số lượng phép toán dấu phẩy động (FLOPs) mà vẫn phải duy trì được độ tin cậy.

Xuất phát từ những phân tích trên, đề tài này đặt ra mục tiêu: Cải tiến kiến trúc Tiny YOLOv3 nhằm nâng cao hiệu năng phát hiện phương tiện và người đi bộ, đồng thời tối ưu hóa để mô hình đủ nhỏ gọn cho việc triển khai thời gian thực trên Raspberry Pi 4. Hướng tiếp cận của đề tài được hiện thực hóa qua ba nhóm giải pháp đột phá:

- Nâng cao độ chính xác: Mở rộng kiến trúc phát hiện đa tỉ lệ kết hợp tích hợp cơ chế chú ý CBAM [8] nhằm tăng cường khả năng trích xuất đặc trưng vùng mục tiêu.
- Tối ưu hóa kiến trúc mạng: Thay thế các lớp tích chập truyền thống trong phần neck và head bằng các khối cấu trúc hiệu quả hơn (C3Ghost [10] và GSConv [9]) nhằm giảm thiểu FLOPs và số lượng tham số.
- Tăng tốc suy diễn: Áp dụng kỹ thuật nén mô hình Tucker Decomposition [12], [14] kết hợp với chuyển đổi định dạng ONNX để tối ưu hóa hiệu suất chạy trên kiến trúc CPU ARM.

1.2 MỤC TIÊU

Mục tiêu tổng quát của đề tài là nghiên cứu, cải tiến và triển khai một hệ thống phát hiện phương tiện và người đi bộ theo thời gian thực dựa trên kiến trúc Tiny YOLOv3, hướng đến việc nâng cao độ chính xác phát hiện so với mô hình gốc và đảm bảo khả năng hoạt động trên thiết bị nhúng Raspberry Pi 4 với tốc độ đáp ứng yêu cầu thực tế. Để đạt được mục tiêu đó, đề tài được triển khai theo ba nhóm mục tiêu cụ thể, tương ứng với ba giai đoạn nghiên cứu và thực nghiệm.

Nhóm mục tiêu thứ nhất tập trung vào việc cải tiến kiến trúc mô hình nhằm tăng độ chính xác phát hiện. Cụ thể, đề tài bổ sung đầu phát hiện thứ ba tại tỉ lệ đặc trưng 52×52 để mô hình có thể nhận diện các đối tượng có kích thước nhỏ — vốn là điểm yếu của Tiny YOLOv3 gốc chỉ sử dụng hai đầu phát hiện. Đồng thời, cơ chế chú ý CBAM (Convolutional Block Attention Module) được tích hợp tại cả ba tỉ lệ đặc trưng (52×52 ,

26×26 và 13×13) nhằm giúp mô hình tập trung vào các vùng ảnh chứa đối tượng và giảm thiểu ảnh hưởng của nhiễu nền. Ngoài ra, số lượng anchor được điều chỉnh từ 6 lên 9 thông qua thuật toán K-means clustering trên tập dữ liệu thực tế, nhằm bao phủ tốt hơn sự đa dạng về kích thước của phương tiện và người đi bộ. Hàm mất mát CIoU [6] cũng được áp dụng thay thế IoU thông thường để tối ưu hóa quá trình hồi quy khung bao đối tượng một cách chính xác hơn.

Nhóm mục tiêu thứ hai hướng đến việc tối ưu hóa kiến trúc mô hình nhằm giảm chi phí tính toán mà không làm suy giảm đáng kể độ chính xác. Phần neck của mô hình — nơi tổng hợp đặc trưng từ các tỉ lệ khác nhau để đưa vào đầu phát hiện — được thay thế bằng cấu trúc C3Ghost, kết hợp giữa mạng CSP (Cross Stage Partial) và các khối GhostBottleneck. Cấu trúc này cho phép tái sử dụng đặc trưng hiệu quả hơn trong khi giảm số phép tính nhờ phép tích chập Ghost rẻ về mặt tính toán. Tại đầu phát hiện 13×13, lớp tích chập thông thường được thay thế bằng GSConv — một phép tích chập kết hợp giữa tích chập chuẩn và tích chập theo chiều sâu với bước trộn kênh (channel shuffle) — nhằm duy trì khả năng trao đổi thông tin đa kênh với chi phí FLOPs thấp hơn. Bên cạnh đó, kỹ thuật Tucker Decomposition được áp dụng để phân tách tensor trọng số của các lớp tích chập lớn trong backbone thành tích của hai lớp tích chập nhỏ hơn, qua đó giảm khoảng 63% số tham số (từ xấp xỉ 7,5 triệu xuống còn khoảng 2,8 triệu tham số).

Nhóm mục tiêu thứ ba là triển khai mô hình đã được cải tiến và nén lên nền tảng Raspberry Pi 4 và đánh giá hiệu năng thực tế. Mô hình sau khi huấn luyện và nén được chuyển đổi sang định dạng ONNX để tận dụng khả năng tối ưu hóa suy diễn trên kiến trúc ARM thông qua ONNX Runtime. Quá trình chuyển đổi bao gồm lượng tử hóa trọng số về INT8 thông qua Post-Training Quantization với tập dữ liệu hiệu chỉnh (calibration), nhằm giảm thêm bộ nhớ sử dụng và tăng tốc tính toán số nguyên trên CPU ARM Cortex-A72 của Raspberry Pi 4. Đề tài đặt mục tiêu đạt được tốc độ suy diễn đủ để ứng dụng trong các kịch bản thực tế như camera giám sát điểm cố định hoặc hệ thống cảnh báo phương tiện tốc độ thấp.

Về phạm vi nghiên cứu, đề tài giới hạn bài toán phát hiện ở hai lớp đối tượng là người đi bộ (person) và phương tiện (car) với kích thước đầu vào cố định 416×416 pixel. Độ chính xác của mô hình được đánh giá theo chỉ số mAP@0.5 theo giao thức Pascal VOC [5] trên tập kiểm tra riêng.

1.3 BỐ CỤC ĐỒ ÁN

Báo cáo đồ án được tổ chức thành sáu chương với nội dung như sau:

Chương 1 — Giới thiệu trình bày bối cảnh và lý do thực hiện đề tài, phân tích những hạn chế của kiến trúc Tiny YOLOv3 gốc khi áp dụng vào bài toán phát hiện phương tiện và người đi bộ trong thực tế. Từ đó, chương này nêu rõ mục tiêu nghiên cứu, phạm vi thực hiện và bố cục tổng thể của báo cáo.

Chương 2 — Cơ sở lý thuyết cung cấp nền tảng kiến thức cần thiết để hiểu các phương pháp được đề xuất trong đề tài. Chương này lần lượt trình bày tổng quan về bài toán phát hiện đối tượng, kiến trúc Tiny YOLOv3, cơ chế chú ý CBAM, các kỹ thuật tích chập hiệu quả bao gồm GhostConv và GSConv, cấu trúc Cross Stage Partial kết hợp GhostBottleneck, và phương pháp nén mô hình Tucker Decomposition.

Chương 3 — Phương pháp đề xuất mô tả chi tiết pipeline cải tiến ba giai đoạn được đề xuất trong đề tài. Giai đoạn đầu trình bày cách thức bổ sung đầu phát hiện thứ ba tại tỉ lệ 52×52 và tích hợp cơ chế chú ý CBAM vào ba vị trí của mô hình. Giai đoạn thứ hai mô tả việc thay thế các lớp tích chập trong neck và head bằng GSConv và C3Ghost V2 nhằm tối ưu hóa chi phí tính toán. Giai đoạn thứ ba trình bày quy trình nén mô hình bằng Tucker Decomposition và chiến lược tinh chỉnh lại sau nén.

Chương 4 — Thực nghiệm và kết quả trình bày môi trường thực nghiệm, mô tả tập dữ liệu sử dụng và thiết lập quá trình huấn luyện. Chương này đưa ra kết quả so sánh định lượng giữa các phiên bản mô hình theo từng giai đoạn cải tiến thông qua chỉ số mAP@0.5, đồng thời phân tích sự thay đổi về số tham số và FLOPs qua từng bước. Phần cuối chương thảo luận về các trường hợp điển hình mô hình xử lý tốt cũng như những trường hợp còn hạn chế.

Chương 5 — Triển khai trên thiết bị nhúng trình bày quy trình chuyển đổi mô hình sang định dạng ONNX và triển khai trên Raspberry Pi 4. Chương này mô tả môi trường triển khai, các bước thực hiện chuyển đổi và lượng tử hóa INT8, sau đó đưa ra kết quả đo lường tốc độ suy diễn thực tế trên thiết bị và nhận xét về tính khả thi của hệ thống trong điều kiện thực tế.

Chương 6 — Kết luận và hướng phát triển tóm tắt những kết quả đã đạt được so với mục tiêu đề ra, chỉ ra các hạn chế còn tồn tại của phương pháp, và đề xuất các hướng nghiên

cứu có thể tiếp tục trong tương lai nhằm hoàn thiện hơn hệ thống phát hiện đối tượng trên thiết bị nhúng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ OBJECT DETECTION

2.1.1 Định nghĩa và phân loại

Phát hiện đối tượng (object detection) là bài toán trong lĩnh vực thị giác máy tính với mục tiêu xác định vị trí và phân loại đồng thời tất cả các đối tượng thuộc các lớp được định nghĩa trước trong một hình ảnh đầu vào. Khác với bài toán phân loại ảnh (image classification) chỉ trả về nhãn cho toàn bộ ảnh, phát hiện đối tượng yêu cầu mô hình dự đoán cả tọa độ của khung bao (bounding box) lẫn nhãn lớp cho từng đối tượng riêng lẻ. Đầu ra của mô hình thường là tập hợp các bộ giá trị gồm tọa độ trung tâm, chiều rộng, chiều cao của khung bao cùng xác suất thuộc từng lớp đối tượng.

Các phương pháp phát hiện đối tượng dựa trên học sâu được chia thành hai nhóm chính. Nhóm thứ nhất là các mô hình hai giai đoạn (two-stage detector), điển hình là họ Faster R-CNN [3], hoạt động theo nguyên tắc: giai đoạn đầu sinh ra các vùng đề xuất (region proposals) có khả năng chứa đối tượng, giai đoạn hai thực hiện phân loại và tinh chỉnh tọa độ cho từng vùng đề xuất đó. Nhóm này đạt độ chính xác cao nhưng tốc độ xử lý chậm do phải thực hiện hai bước độc lập. Nhóm thứ hai là các mô hình một giai đoạn (one-stage detector), điển hình là họ YOLO (You Only Look Once) [1], thực hiện dự đoán vị trí và lớp đối tượng trực tiếp từ toàn bộ ảnh trong một lần lan truyền xuôi duy nhất. Nhờ vậy, các mô hình một giai đoạn có tốc độ nhanh hơn đáng kể, phù hợp cho các ứng dụng yêu cầu xử lý thời gian thực.

2.1.2 Anchor box

Hầu hết các mô hình phát hiện đối tượng hiện đại sử dụng cơ chế anchor box — các khung bao tham chiếu được xác định trước với nhiều tỉ lệ khác nhau — để hỗ trợ quá trình dự đoán vị trí đối tượng. Thay vì dự đoán trực tiếp tọa độ tuyệt đối của khung bao, mô hình học cách dự đoán độ lệch (offset) giữa khung bao thực và anchor gần nhất. Phương pháp này giúp ổn định quá trình huấn luyện và cải thiện độ chính xác vị trí, đặc biệt với các đối tượng có hình dạng và kích thước đa dạng.

Các giá trị anchor thường được xác định bằng thuật toán K-means clustering trên tập dữ liệu huấn luyện, nhằm đảm bảo các anchor phản ánh đúng phân phối kích thước của

đối tượng trong từng bài toán cụ thể. Trong đề tài này, chín anchor được phân nhóm thành ba tập tương ứng với ba tỉ lệ đặc trưng, trong đó anchor lớn phụ trách đặc trưng 13×13 để phát hiện đối tượng lớn, anchor trung bình phụ trách đặc trưng 26×26 , và anchor nhỏ phụ trách đặc trưng 52×52 để phát hiện đối tượng nhỏ.

2.1.3 Chỉ số IoU và CIoU Loss

Intersection over Union (IoU) là chỉ số đo mức độ chồng lấp giữa khung bao dự đoán và khung bao thực tế (ground truth). IoU được tính bằng tỉ lệ giữa diện tích phần giao và diện tích phần hợp của hai khung bao:

$$IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|}, \quad (2.1)$$

Trong đó:

- B_{pred} : khung bao dự đoán của mô hình (predicted bounding box);
- B_{gt} : khung bao thực tế (ground-truth bounding box);
- $B_{pred} \cap B_{gt}$: diện tích vùng giao nhau (intersection) giữa hai khung bao;
- $B_{pred} \cup B_{gt}$: tổng diện tích vùng hợp (union) của hai khung bao.

Giá trị IoU nằm trong khoảng $[0, 1]$, trong đó giá trị càng gần 1 cho thấy khung bao dự đoán càng khớp với thực tế. Tuy nhiên, hàm mất mát dựa trên IoU thông thường có nhược điểm là không cung cấp gradient khi hai khung bao không có phần giao, và không xét đến hình dạng cũng như hướng tương đối giữa hai khung bao.

Để khắc phục các hạn chế trên, đề tài sử dụng hàm mất mát CIoU (Complete IoU) [6]. CIoU bổ sung hai thành phần so với IoU thông thường: khoảng cách giữa tâm hai khung bao được chuẩn hóa theo đường chéo của khung bao bao ngoài nhỏ nhất chứa cả hai, và hệ số đo sự đồng nhất về tỉ lệ chiều rộng–chiều cao giữa khung bao dự đoán và ground truth. Công thức của hàm loss CIoU:

$$L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v, \quad (2.2)$$

Trong đó:

- L_{CIoU} : Giá trị của hàm mất mát CIoU. Giá trị này càng nhỏ (gần 0) thì hộp dự đoán càng sát với hộp thực tế.

- $1 - IoU$: Thành phần đánh giá diện tích chồng lấp. IoU là tỷ lệ giữa phần diện tích giao nhau và phần diện tích hợp nhất của hai hộp.
- $\frac{\rho^2(b, b^{gt})}{c^2}$: Thành phần đánh giá khoảng cách tâm.
 - o b, b^{gt} : Lần lượt là tọa độ tâm của hộp dự đoán và hộp thực tế.
 - o $\rho^2(\dots)$: Bình phương khoảng cách Euclidean giữa hai tâm này.
 - o c : chiều dài đường chéo của bounding box nhỏ nhất có thể bao trọn cả hai hộp dự đoán và thực tế. Việc chia cho c^2 giúp chuẩn hóa khoảng cách này về khoảng $[0,1]$.
- av : Thành phần đánh giá sự nhất quán về tỷ lệ khung hình (Aspect Ratio). Đây là điểm cải tiến lớn nhất của CIOU.

Nhờ vậy, CIOU cung cấp tín hiệu gradient phong phú hơn trong suốt quá trình huấn luyện, giúp mô hình hội tụ nhanh hơn và dự đoán vị trí khung bao chính xác hơn so với các hàm mất mát IoU truyền thống.

2.1.4 Phát hiện đa tỉ lệ

Một trong những thách thức cốt lõi của bài toán phát hiện đối tượng là sự đa dạng về kích thước: cùng một loại đối tượng có thể xuất hiện với kích thước rất khác nhau tùy theo khoảng cách đến camera. Phát hiện đa tỉ lệ (multi-scale detection) giải quyết vấn đề này bằng cách sử dụng các đặc trưng ở nhiều mức độ phân giải không gian khác nhau trong mạng nơ-ron. Các đặc trưng có độ phân giải thấp (như 13×13) mang thông tin ngữ nghĩa cao, phù hợp để phát hiện các đối tượng lớn. Ngược lại, các đặc trưng có độ phân giải cao (như 52×52) giữ lại nhiều thông tin chi tiết không gian, phù hợp để phát hiện các đối tượng nhỏ.

Kiến trúc Feature Pyramid Network (FPN) [4] là một trong những cách tiếp cận phổ biến để tổng hợp thông tin đa tỉ lệ bằng cách kết hợp đặc trưng từ nhiều tầng của backbone thông qua các kết nối bỏ qua (skip connections) và phép upsample. Tiny YOLOv3 gốc sử dụng hai đầu phát hiện tại 13×13 và 26×26 , tuy nhiên thiếu đầu phát hiện tại 52×52 khiến mô hình bỏ sót nhiều đối tượng nhỏ. Đề tài này bổ sung thêm đầu phát hiện thứ ba tại 52×52 để giải quyết hạn chế này.

2.1.5 Đánh giá mô hình – $mAP@0.5$

Mean Average Precision tại ngưỡng IoU 0.5 ($mAP@0.5$) là chỉ số đánh giá phổ biến nhất cho bài toán phát hiện đối tượng theo giao thức Pascal VOC [5]. Để tính $mAP@0.5$, với mỗi lớp đối tượng, các dự đoán được sắp xếp theo thứ tự giảm dần của độ tin cậy (confidence score). Một dự đoán được coi là đúng (True Positive) nếu IoU với ground truth tương ứng đạt từ 0.5 trở lên và lớp dự đoán khớp với lớp thực tế. Từ đó, đường cong Precision–Recall được vẽ và diện tích dưới đường cong này chính là Average Precision (AP) của lớp đó. $mAP@0.5$ là giá trị trung bình của AP trên tất cả các lớp đối tượng trong bài toán.

Chỉ số này phản ánh đồng thời hai khía cạnh của mô hình: khả năng tìm đúng đối tượng (Recall) và độ chính xác của các dự đoán trả về (Precision). Một mô hình tốt cần đồng thời đạt Precision và Recall cao, tức là ít bỏ sót và ít dự đoán sai. Trong đề tài này, $mAP@0.5$ được sử dụng làm chỉ số đánh giá chính để so sánh các phiên bản mô hình qua từng giai đoạn cải tiến.

2.2 KIẾN TRÚC TINY YOLOV3

2.2.1 Tổng quan về Tiny YOLOv3

Tiny YOLOv3 [2] là phiên bản tối giản của kiến trúc YOLOv3, được tinh chỉnh chuyên biệt để triển khai trên các thiết bị Edge AI và hệ thống nhúng có tài nguyên phần cứng giới hạn. Trong khi YOLOv3 tiêu chuẩn sử dụng bộ khung (backbone) Darknet-53 đồ sộ với khoảng 61 triệu tham số, Tiny YOLOv3 đã cắt giảm mạnh mẽ cấu trúc này, đưa tổng số tham số xuống chỉ còn khoảng 8,7 triệu. Sự đánh đổi này (giảm số tầng, giảm số kênh, và chỉ sử dụng hai nhánh phát hiện thay vì ba) giúp mô hình đạt tốc độ suy luận thời gian thực (real-time) cực nhanh mà vẫn duy trì được độ chính xác ở mức chấp nhận được. Kiến trúc tinh gọn này chính là nền tảng cốt lõi để đề tài kế thừa và phát triển.

2.2.2 Khối tích chập cơ bản (*DarknetConv2D_BN_Leaky*)

Đơn vị tính toán xuyên suốt làm nên sức mạnh của Tiny YOLOv3 là khối *DarknetConv2D_BN_Leaky*. Khối này bao gồm ba thành phần xếp nối tiếp nhau:

- Conv2D: Lớp tích chập thực hiện trích xuất đặc trưng. Lớp này không sử dụng tham số bias do vai trò dịch chuyển đã được xử lý ở bước tiếp theo.

- Batch Normalization (BN) [7]: Chuẩn hóa đầu ra về phân phối chuẩn, giúp mô hình tăng tốc độ hội tụ và ổn định dòng chảy gradient trong quá trình huấn luyện.
- Leaky ReLU: Hàm kích hoạt phi tuyến tính. Khác với ReLU thông thường sẽ triệt tiêu hoàn toàn các nơ-ron mang giá trị âm (gây ra hiện tượng "dying ReLU"), Leaky ReLU cho phép một lượng nhỏ gradient đi qua, được định nghĩa bằng công thức:

$$f(x) = \begin{cases} x, & \text{nếu } x \geq 0 \\ 0.1x, & \text{nếu } x < 0 \end{cases}, \quad (2.3)$$

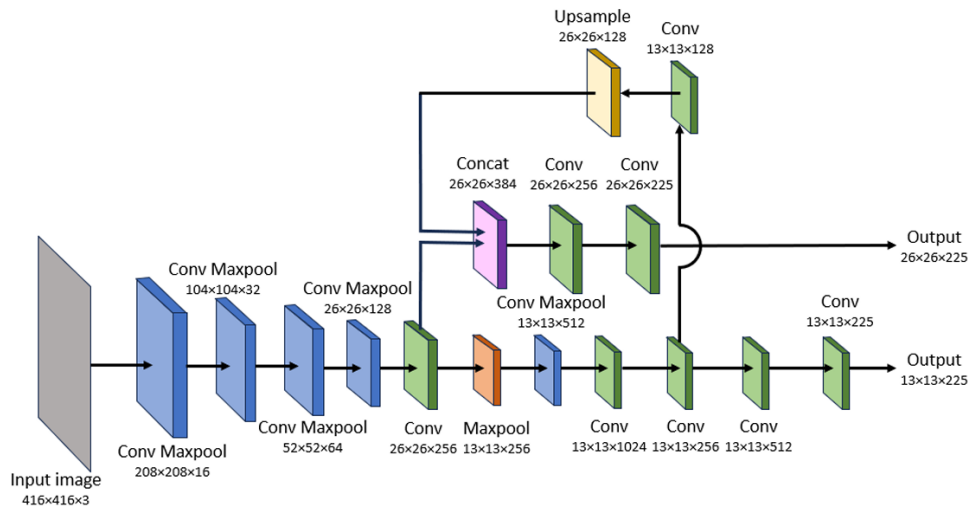
Trong đó:

- $f(x)$: Giá trị đầu ra của nơ-ron sau khi đi qua hàm kích hoạt.
- x : Giá trị đầu vào của nơ-ron nhận được sau khi qua lớp tích chập và chuẩn hóa Batch Normalization.

Trường hợp $x \geq 0$, hàm sẽ trả về đúng giá trị nguyên bản đó. Điểm này giúp tín hiệu và luồng gradient được truyền đi mạnh mẽ, tuyến tính ở nửa không gian dương

Trường hợp $x < 0$, thay vì triệt tiêu hoàn toàn thành số 0 như hàm ReLU truyền thống, hàm sẽ nhân giá trị đó với một hệ số rất nhỏ (ở đây là 0.1).

2.2.3 Kiến trúc trích xuất đặc trưng (Backbone)



Hình 2.1: Sơ đồ kiến trúc luồng dữ liệu của mạng Tiny YOLOv3 nguyên bản.

Dựa trên sơ đồ luồng dữ liệu, backbone của Tiny YOLOv3 thực hiện quá trình hạ lấy mẫu (downsampling) qua một chuỗi các khối tích chập xen kẽ với các lớp MaxPool. Với ảnh đầu vào có độ phân giải $416 \times 416 \times 3$, quá trình trích xuất diễn ra như sau:

- Dữ liệu đi qua 4 khối *Conv* + *MaxPool* liên tiếp, khiến kích thước không gian giảm đi một nửa và số kênh nhân đôi qua mỗi bước: $208 \times 208 \times 16 \rightarrow 104 \times 104 \times 32 \rightarrow 52 \times 52 \times 64 \rightarrow 26 \times 26 \times 128$.
- Tiếp theo, dữ liệu đi qua lớp Conv2D để tạo ra bản đồ đặc trưng $26 \times 26 \times 256$ (ký hiệu là f_3). Tại đây, mô hình tạo ra điểm rẽ nhánh đầu tiên để lưu lại thông tin không gian chi tiết phục vụ cho phần sau.
- Nhánh chính tiếp tục đi qua lớp MaxPool và các lớp Conv2D (với số kênh tăng lên 1024, rồi hạ xuống 256) để tạo ra bản đồ đặc trưng $13 \times 13 \times 256$ (ký hiệu là f_2). Điểm đặc biệt ở tầng này là lớp MaxPool sử dụng bước trượt (stride) bằng 1 và padding 'same' nhằm giữ nguyên độ phân giải không gian 13×13 .

2.2.4 Cổ mạng (Neck) và Hai nhánh phát hiện (Detection Heads)

Kiến trúc sử dụng hai đầu phát hiện song song để nhận diện đối tượng ở các kích thước khác nhau. Trong phạm vi đề tài này (chỉ phát hiện 2 lớp: người và xe ô tô), số lượng kênh đầu ra được tinh chỉnh lại thành 21 kênh tại mỗi ô lưới, tương ứng với công thức:

$$3 \text{ anchors} \times (5 \text{ bounding box} + 2 \text{ classes}) = 21$$

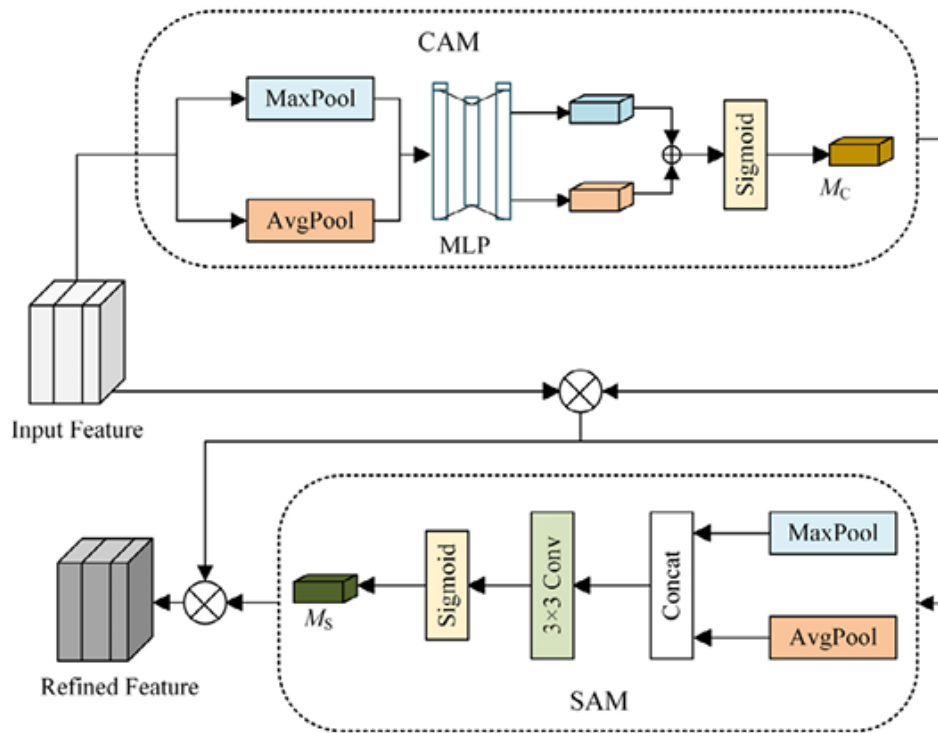
Nhánh phát hiện 1 (Head 1 - Kích thước 13×13): Nhận đầu vào là tensor $13 \times 13 \times 256$ (f_2). Dữ liệu được đưa qua khối Conv2D mở rộng lên 512 kênh, sau đó đi qua lớp tích chập 1×1 cuối cùng để xuất ra tensor $13 \times 13 \times 21$. Nhánh này chịu trách nhiệm sử dụng 3 anchor box lớn để phát hiện các đối tượng có kích thước lớn (như xe ô tô) trên khung hình.

Nhánh phát hiện 2 (Head 2 - Kích thước 26×26): Để phát hiện các đối tượng nhỏ hơn, mô hình trích xuất một phần dữ liệu từ tensor $13 \times 13 \times 256$, cho đi qua lớp Conv2D 128 kênh và thực hiện nội suy (Upsample 2x) để phóng to thành $26 \times 26 \times 128$. Sau đó, tensor này được ghép nối với nhánh f_3 ($26 \times 26 \times 256$) được bảo lưu từ backbone, tạo thành tensor kết hợp $26 \times 26 \times 384$. Dữ liệu kết hợp này đi qua một lớp Conv2D 256 kênh, và cuối cùng là lớp tích chập 1×1 để xuất ra tensor $26 \times 26 \times 21$. Phép ghép nối này hoạt động tương tự cơ chế FPN (Feature Pyramid Network), giúp bù đắp thông tin không gian đã mất cho các đối tượng vừa hoặc nhỏ.

2.3 CƠ CHẾ ATTENTION – CBAM

2.3.1 Tổng quan về CBAM

Trong các bài toán thị giác máy tính như phát hiện người và phương tiện, bối cảnh nền phức tạp thường gây nhiễu cho quá trình trích xuất đặc trưng. Để giải quyết vấn đề này, module chú ý CBAM [8] được tích hợp nhằm giúp mạng chắt lọc các vùng thông tin quan trọng. CBAM tinh chỉnh feature map đầu vào bằng cách áp dụng tuần tự hai khối module: Channel Attention (xác định kênh đặc trưng nào quan trọng) và Spatial Attention (xác định vùng không gian nào quan trọng).



Hình 2.2: Kiến trúc tổng thể của module CBAM (Channel Attention Module và Spatial Attention Module)

2.3.2 Module Channel Attention (Chú ý theo Kênh)

Khối này đánh giá mức độ quan trọng của từng kênh thông tin độc lập. Dữ liệu đầu vào F được xử lý qua hai phép toán Global Average Pooling và Global Max Pooling để thu thập cả thông tin tổng quát lẫn thông tin nổi bật nhất của bức ảnh. Cả hai kết quả này đi

qua chung một mạng nơ-ron đa tầng (Shared MLP) với hệ số giảm r (thường $r=8$) nhằm tối ưu chi phí tính toán.

Công thức tính toán bản đồ chú ý theo kênh $M_c \in \mathbb{R}^{C \times 1 \times 1}$:

$$M_c(F) = \sigma \left(\text{MLP}(\text{AvgPool}(F)) + \text{MLP}(\text{MaxPool}(F)) \right), \quad (2.4)$$

Trong đó:

- **M_c** : Bản đồ chú ý theo kênh (Channel Attention Map) ở đầu ra, biểu diễn mức độ quan trọng (trọng số) của từng kênh đặc trưng. Tensor này thường có kích thước $C \times 1 \times 1$
- **F** : Bản đồ đặc trưng (Feature Map) đầu vào thu được từ các lớp tích chập trước đó, có kích thước $C \times H \times W$
- σ : Hàm kích hoạt Sigmoid, chuẩn hóa đầu ra về khoảng $[0, 1]$ để làm hệ số tỷ lệ: giá trị gần 1 giúp nhấn mạnh kênh quan trọng, gần 0 giúp triệt tiêu kênh nhiễu.
- $\text{AvgPool}(F)$ và $\text{MaxPool}(F)$: Phép gộp trung bình (thu thập bối cảnh tổng quát) và gộp cực đại (trích xuất đặc trưng nổi bật nhất) trên mỗi kênh.
 - AvgPool giúp hệ thống thu thập thông tin bối cảnh tổng quát của bức ảnh.
 - MaxPool giúp hệ thống tập trung vào các đặc trưng sắc nét, nổi bật nhất (ví dụ như đường nét, góc cạnh củ đối tượng).
- **MLP** : Mạng nơ-ron đa tầng dùng chung trọng số để đánh giá thông tin từ hai nhánh gộp trên.

Đầu ra của module này là feature map F' đã được nhân tỷ lệ (scale) theo trọng số của từng kênh:

$$F' = M_c(F) \otimes F, \quad (2.5)$$

2.3.3 Module Spatial Attention (Chú ý theo Không gian)

Sau khi có feature map được hiệu chỉnh theo kênh F' , khối Spatial Attention tiếp tục đánh giá mức độ quan trọng về mặt vị trí. Dữ liệu được gộp theo chiều kênh bằng AvgPool và MaxPool , tạo ra hai bản đồ đặc trưng 2D. Chúng được nối kết lại thành một tensor chung và đi qua một lớp tích chập với kích thước 7×7 nhằm đảm bảo tầm nhìn đủ lớn để nắm bắt ngữ cảnh.

Công thức tính toán bản đồ chú ý theo không gian $M_s \in \mathbb{R}^{1 \times H \times W}$:

$$M_s(F') = \sigma(\text{Conv}^{7 \times 7}([\text{AvgPool}(F'); \text{MaxPool}(F')])), \quad (2.6)$$

Trong đó:

- $M_s(F')$: Bản đồ trọng số chú ý theo không gian.
- F' : Bản đồ đặc trưng đầu vào.
- $\text{AvgPool}(F)$ và $\text{MaxPool}(F)$: Phép gộp trung bình và gộp cực đại thực hiện dọc theo chiều kênh, giúp nén thông tin lại thành hai mặt phẳng 2D.
 - $[\text{AvgPool}(F'); \text{MaxPool}(F')]$: Phép ghép nối hai mặt phẳng 2D vừa tạo thành một khối dữ liệu duy nhất.
 - $\text{Conv}^{7 \times 7}$: Lớp tích chập với kích thước cửa sổ 7×7 , cung cấp trường nhìn đủ rộng để nắm bắt ngữ cảnh không gian xung quanh mỗi điểm ảnh.
- σ : Hàm kích hoạt Sigmoid, chuẩn hóa kết quả về khoảng $[0, 1]$ để tạo thành một "mặt nạ" không gian: các vị trí có giá trị gần 1 sẽ được làm nổi bật nơi có đối tượng, các vị trí gần 0 sẽ bị triệt tiêu các loại nhiễu nền.

Cuối cùng tạo ra 1 feature map bằng cách nhân bản đồ trọng số với F' :

$$F'' = M_s(F') \otimes F', \quad (2.7)$$

2.3.4 Thực nghiệm tính hiệu quả của CBAM

Trong quá trình trích xuất đặc trưng, không phải mọi thông tin trên feature map đều có giá trị như nhau. Để đơn giản hóa việc phân tích, giả sử feature map đầu vào gồm hai thành phần:

$$F = F_{obj} + F_{bg}, \quad (2.8)$$

Trong đó:

- F_{obj} : đặc trưng của đối tượng (người, phương tiện)
- F_{bg} : đặc trưng của nền

Khi không sử dụng cơ chế chú ý, đầu ra của mạng được biểu diễn như (2.8). Điều này cho thấy đặc trưng của đối tượng và nền đều được truyền sang các lớp tiếp theo với mức độ như nhau, làm giảm khả năng phân biệt đối tượng trong các bối cảnh phức tạp.

Khi tích hợp CBAM, module sẽ tự động học các trọng số chú ý thông qua quá trình huấn luyện bằng thuật toán lan truyền ngược. Để thuận tiện trong phân tích, giả sử trọng số học được đối với vùng đối tượng là α và vùng nền là β với:

$$0 < \beta < \alpha \leq 1$$

Khi đó, đầu ra của CBAM được biểu diễn:

$$F_{out} = \alpha F_{obj} + \beta F_{bg}, \quad (2.9)$$

Do $\alpha > \beta$, đặc trưng của đối tượng được tăng cường trong khi đặc trưng của nền bị suy giảm. Xét tỷ lệ giữa đặc trưng của đối tượng và đặc trưng của nền trước khi áp dụng CBAM:

$$R = \frac{F_{obj}}{F_{bg}}, \quad (2.10)$$

Sau khi áp dụng CBAM, tỷ lệ này trở thành:

$$\begin{aligned} R' &= \frac{\alpha F_{obj}}{\beta F_{bg}} \\ \Rightarrow R' &= \frac{\alpha}{\beta} \times \frac{F_{obj}}{F_{bg}} \end{aligned}$$

Từ (2.10), suy ra:

$$\Rightarrow R' = \frac{\alpha}{\beta} \times R$$

Mà $\alpha > \beta \Rightarrow \frac{\alpha}{\beta} > 1$ nên :

$$R' > R$$

Kết quả trên chứng minh rằng CBAM làm tăng tỷ lệ giữa đặc trưng của đối tượng và đặc trưng của nền, giúp mô hình tập trung vào các thông tin quan trọng và giảm ảnh hưởng của nhiễu. Nhờ đó, khả năng biểu diễn đặc trưng được cải thiện, góp phần nâng cao độ chính xác phát hiện đối tượng trong khi chỉ làm tăng rất ít chi phí tính toán.

2.4 GROUPED SHUFFLE CONVOLUTION

2.4.1 Hạn chế của tích chập tiêu chuẩn và động lực đề xuất

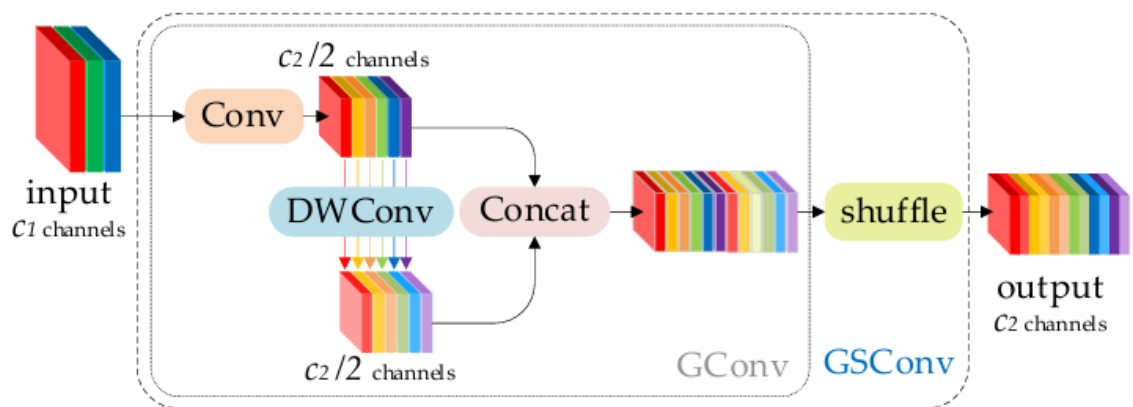
Trong phần cổ (Neck) của các mô hình phát hiện đối tượng đa tầng, các lớp tích chập thường sử dụng số lượng kênh (channel) rất lớn, ví dụ 256 hoặc 512, nhằm bảo toàn tối đa thông tin ngữ nghĩa được truyền từ khung xương (Backbone). Tuy nhiên, một phép tích chập tiêu chuẩn (Standard Convolution) đòi hỏi khối lượng tính toán (FLOPs) rất cao.

Khi áp dụng trên các thiết bị nhúng giới hạn tài nguyên, các lớp tích chập nặng nề này thường trở thành nút thắt cổ chai, làm suy giảm tốc độ suy luận.

Để giải quyết vấn đề này, phép tích chập theo chiều sâu (Depthwise Convolution - DWConv) thường được dùng thay thế nhờ chi phí tính toán cực rẻ. Mặc dù vậy, DWConv lại bộc lộ nhược điểm chí mạng là không có khả năng trao đổi thông tin chéo giữa các kênh, làm giảm sút năng lực học các đặc trưng phức tạp. Khối GSConv [9] ra đời như một giải pháp dung hòa tối ưu: vừa kế thừa tính năng trao đổi thông tin đa kênh của tích chập tiêu chuẩn, vừa tận dụng tốc độ của tích chập theo chiều sâu.

2.4.2 Kiến trúc và nguyên lý hoạt động của GSConv

Khối GSConv thực hiện xử lý dữ liệu đầu vào qua ba bước chính nhằm sinh ra tập đặc trưng đa dạng nhưng tối ưu về chi phí:



Hình 2.3: Kiến trúc tổng thể của khối GSConv (Grouped Shuffle Convolution)

- Bước 1: Áp dụng một phép tích chập tiêu chuẩn lên toàn bộ C_{in} kênh đầu vào, nhưng chỉ dùng một nửa số bộ lọc ($C_{out}/2$). Hành động này giúp trích xuất được các thông tin chéo giữa các kênh (cross-channel) với chi phí tính toán chỉ bằng một nửa so với thông thường. Tập đặc trưng này được gọi là x_1 .
- Bước 2: Lấy x_1 làm đầu vào, tiếp tục áp dụng phép tích chập theo chiều sâu (DWConv) với kích thước cửa sổ 3×3 để sinh ra tập đặc trưng thứ hai x_2 . Bước này bổ sung thêm thông tin bối cảnh không gian cục bộ với chi phí tính toán gần như không đáng kể.

- Bước 3: Sau khi có 2 tập đặc trưng đều có kích thước bằng nhau ($C_{out}/2$) tiến hành ghép nối dọc theo chiều kênh để tạo ra tensor z có đúng kích thước mục tiêu C_{out} .

$$z = \text{Concat}(x_1, x_2) \in \mathbb{R}^{H \times W \times C_{out}}, \quad (2.11)$$

Trong đó:

- z : bản đồ đặc trưng tổng hợp sau khi thực hiện ghép nối.
- Concat : Phép toán ghép nối dữ liệu theo chiều dọc
- x_1 : Tập đặc trưng thu được từ phép tích chập tiêu chuẩn ở bước 1, chứa thông tin giao thoa chéo giữa các kênh.
- x_2 : Tập đặc trưng thu được từ phép tích chập theo chiều sâu ở bước 2, chứa thông tin bối cảnh không gian cục bộ.
- $\mathbb{R}^{H \times W \times C_{out}}$: Không gian tensor của kết quả đầu ra.

2.4.3 Cơ chế Channel Shuffle

Sau khi thực hiện xong phép ghép nối trên, cấu trúc tensor sẽ bị chia ra 2 nửa: nửa đầu hoàn toàn là dữ liệu từ nhánh tích chập tiêu chuẩn, nửa sau hoàn toàn từ nhánh DWConv. Sự phân tách này làm ngăn cản các lớp xử lý phía sau tương tác đồng thời với cả hai loại thông tin.

Để giải quyết vấn đề này, cơ chế Channel Shuffle được áp dụng. Kỹ thuật này sắp xếp lại vị trí các kênh theo một quy tắc hoán vị cố định:

$$[0, C/2, 1, C/2 + 1, 2, C/2 + 2, \dots, C/2 - 1, C - 1]$$

- Các chỉ số $(0, 1, 2, \dots, C/2 - 1)$: đại diện cho các kênh thuộc nửa đầu tensor, tập đặc trưng x_1 sinh ra từ nhánh tích chập tiêu chuẩn.
- Các chỉ số $(C/2, C/2 + 1, \dots, C - 1)$: Đại diện cho các kênh thuộc nửa sau tensor, tập đặc trưng x_2 sinh ra từ nhánh tích chập theo chiều sâu.

Việc hoán vị kênh chỉ là một phép thao tác bộ nhớ (gather), không đòi hỏi thêm tham số học nào, nhưng mang lại hiệu quả lớn. Nó đảm bảo đặc trưng từ hai nhánh được đan xen đồng đều, giúp duy trì dòng chảy thông tin mượt mà và tăng cường khả năng biểu diễn của mạng nơ-ron.

2.4.4 Hiệu quả cắt giảm tính toán của phương pháp

Việc thay thế một phần tích chập tiêu chuẩn bằng DWConv giúp GSConv tiết kiệm đáng kể chi phí FLOPs so với khối Conv2D truyền thống có cùng cấu hình đầu ra:

$$FLOPs_{GSConv} \approx \frac{H \cdot W \cdot C_{out}}{2} \cdot (C_{in} \cdot k^2 + 9), \quad (2.12)$$

Trong đó:

- $FLOPs_{GSConv}$: Tổng số lượng phép toán dấu phẩy động, đại diện cho khối lượng tính toán của toàn bộ khối GSConv.
- H, W : Lần lượt là chiều cao và chiều rộng của bản đồ đặc trưng.
- C_{in}, C_{out} : Số lượng kênh đặc trưng ở đầu vào và tổng số lượng kênh ở đầu ra.
- k^2 : Kích thước không gian của bộ lọc trong phép tích chập tiêu chuẩn (kernel $k \times k$).
- $\frac{H \cdot W \cdot C_{out}}{2}$: Nhân tử chung biểu thị việc các phép toán chỉ áp dụng để sinh ra hoặc xử lý một nửa số lượng kênh đầu ra ($C_{out}/2$).

Ví dụ, đối với lớp xử lý có 512 kênh đầu vào và đầu ra, GSConv giúp tiết kiệm khoảng 45–50% khối lượng tính toán mà thông tin chéo giữa các kênh vẫn được bảo toàn.

2.4.5 Thực nghiệm trên số để so sánh GSConv và Standard Conv

Chi phí tính toán của Standard Conv phụ thuộc vào kích thước bản đồ đặc trưng và số lượng kênh, được tính theo công thức:

$$FLOPs_{std} = H \cdot W \cdot C_{in} \cdot C_{out} \cdot k^2, \quad (2.13)$$

Trong đó:

- H, W : lần lượt là chiều cao và chiều rộng của bản đồ đặc trưng.
- C_{in}, C_{out} : lần lượt là số lượng kênh đầu vào và đầu ra.
- k : kích thước bộ lọc (kernel size), thường $k = 3$.

Chi phí tính toán của GSConv chỉ sử dụng tích chập tiêu chuẩn để trích xuất một nửa số kênh đầu ra ($C_{out}/2$), sau đó sử dụng Depthwise Conv với bộ lọc 3×3 để sinh ra một nửa còn lại được thể hiện qua công thức (2.9).

Từ đó ta lập được tỉ lệ FLOPs của 2 lớp tích chập:

$$\frac{FLOPs_{GSConv}}{FLOPs_{std}} = \frac{\frac{H \cdot W \cdot C_{out}}{2} \cdot (C_{in} \cdot k^2 + 9)}{H \cdot W \cdot C_{in} \cdot C_{out} \cdot k^2}$$

Thực hiện triệt tiêu các đại lượng chung (H, W, C_{out}) ở tử và mẫu số, ta thu được công thức:

$$\frac{FLOPs_{GSConv}}{FLOPs_{std}} = \frac{1}{2} \left(1 + \frac{9}{C_{in} \cdot k^2} \right)$$

Trong các mạng nơ-ron thực tế, tích số $C_{in} \cdot k^2$ luôn mang giá trị rất lớn so với 9. (Ví dụ: Tại lớp Head 1 của mô hình có $C_{in} = 256, k = 3 \Rightarrow C_{in} \cdot k^2 = 2304 \gg 9$). Do đó, phân số $\frac{9}{C_{in} \cdot k^2}$ sẽ tiến rất gần về 0. Từ đó suy ra:

$$\frac{FLOPs_{GSConv}}{FLOPs_{std}} \approx \frac{1}{2}$$

Sự triệt tiêu khối lượng tính toán FLOPs của phương pháp này xuất phát từ sự khác biệt trong số lượng phép nhân chập tại mỗi vị trí không gian. Để minh họa, giả sử bản đồ đặc trưng có đầu vào gồm 9 kênh ($C_{in} = 9$) và sử dụng khối kernel 3×3 :

- Standard Conv: Bộ lọc phải xử lý đâm xuyên qua toàn bộ độ sâu 9 kênh cùng lúc. Tại mỗi vị trí không gian, hệ thống phải thực hiện $3 \times 3 \times 9 = 81$ phép nhân
- Depthwise Conv: Bộ lọc chỉ tính toán độc lập trên bề mặt 2D của từng kênh riêng biệt. Do đó, tại mỗi vị trí tương ứng, hệ thống chỉ cần thực hiện $3 \times 3 \times 1 = 9$ phép nhân.

Chính nhờ sự tối ưu này, lượng phép tính (FLOPs) phát sinh thêm khi sử dụng Depthwise Conv để nội suy một nửa số kênh còn lại trong module GSConv là cực kỳ nhỏ, gần như không đáng kể so với chi phí tính toán nguyên bản của Standard Conv.

2.5 CROSS STAGE PARTIAL, GHOST-BOTTLENECK VÀ C3GHOST

2.5.1 Tổng quan về sự dư thừa trong bản đồ đặc trưng

Trong các mạng nơ-ron tích chập sâu, nhiều bản đồ đặc trưng ở cùng một lớp thường chứa lượng thông tin tương tự nhau, chỉ khác biệt thông qua các phép biến đổi nhỏ như lật

gương hoặc dịch chuyển. Sự dư thừa này chính là nguyên nhân gây lãng phí lớn về tài nguyên tính toán. Để giải quyết bài toán này, kiến trúc mạng GhostNet [10] đã được đề xuất nhằm thay thế phép tích chập thông thường bằng một chiến lược tính toán tiết kiệm hơn mà vẫn đảm bảo số lượng đặc trưng cần thiết.

2.5.2 Tích chập Ghost (GhostConv)

GhostConv là khối xử lý cốt lõi của kiến trúc GhostNet, hoạt động dựa trên nguyên lý phân chia quá trình trích xuất đặc trưng thành hai bước tính gọn:

- Bước 1: Thay vì tính toán toàn bộ số kênh đầu ra, hệ thống sử dụng một phép tích chập tiêu chuẩn với kích thước hạt nhân 1×1 để sinh ra một nửa số lượng kênh ($C_{out}/2$)
- Bước 2: Nửa số lượng kênh còn lại được nội suy bằng cách áp dụng phép tích chập theo chiều sâu (depthwise convolution) với kích thước 3×3 lên tập đặc trưng vừa được tạo ở Bước 1.

Sau hai bước đó, hai tập đặc trưng trên được ghép nối với nhau tạo thành đầu ra hoàn chỉnh gồm C_{out} kênh. Đầu ra của khối GhostConv được biểu diễn theo công thức:

$$y = \text{Concat}(x_1, x_2) \in \mathbb{R}^{H \times W \times C_{out}}, \quad (2.14)$$

Trong đó:

- y : đầu ra là bản đồ đặc trưng của khối GhostConv.
- Concat : phép ghép nối dọc theo kênh.
- x_1 : tập đặc trưng gốc được sinh ra từ phép tích chập tiêu chuẩn ở bước 1.
- x_2 : tập đặc trưng ghost features được sinh ra từ các phép tích chập theo chiều sâu (depthwise convolution) ở bước 2.
- $\mathbb{R}^{H \times W \times C_{out}}$: Kích thước không gian của tensor đầu ra (mang giá trị số thực), bao gồm Chiều cao H , Chiều rộng W và Tổng số kênh đầu ra C_{out} .
 - C_{out} : tổng số kênh khi gộp x_1 và x_2 .

Bằng việc giảm kích thước xuống 1×1 ở bước đầu và áp dụng tích chập theo chiều sâu (depthwise convolution) ở bước sau, GhostConv giúp giảm thiểu xấp xỉ một nửa khối lượng tính toán (FLOPs) so với phép tích chập 3×3 tiêu chuẩn.

Kiểm chứng khối lượng tính toán của GhostConv so với Standard Conv:

Khối lượng phép tính của Standard Conv được biểu diễn bởi công thức (). Còn khối lượng tính toán của GhostConv được biểu diễn bởi công thức:

$$FLOPs_{Ghost} = H \times W \times C_{in} \times \frac{C_{out}}{s} \times k^2, \quad (2.15)$$

Với s là hệ số Ghost ($s > 1$). Lập tỷ lệ so sánh giữa hai phương pháp:

$$\frac{FLOPs_{Ghost}}{FLOPs_{Conv}} = \frac{1}{s}$$

Do $s > 1$ nên:

$$FLOPs_{Ghost} < FLOPs_{Conv}$$

Ví dụ, khi $s = 2$:

$$FLOPs_{Ghost} = \frac{1}{2} FLOPs_{Conv}$$

Điều này cho thấy Ghost Module chỉ cần khoảng 50% chi phí tính toán so với tích chập thông thường nhưng vẫn tạo được cùng số lượng feature map.

2.5.3 Khối cổ chai Ghost (GhostBottleneck)

Khối GhostBottleneck được xây dựng dựa trên thiết kế của cấu trúc residual (trường mạng ResNet) nhưng thay thế toàn bộ các lớp tích chập tiêu chuẩn bằng phép toán GhostConv:

- Lớp GhostConv đầu tiên đóng vai trò mở rộng số lượng kênh từ C_{in} lên C_{mid} . Mô hình trong đề tài này được cấu hình với $C_{mid} = C_{out}/2$ nhằm kiểm soát và giảm thiểu chi phí tính toán.
- Lớp GhostConv thứ hai thực hiện thu hẹp số kênh về lại kích thước C_{out} .

Khối này tạo ra một đường vòng để cộng dữ liệu đầu vào vào kết quả ở đầu ra. Đường này đóng vai trò như một luồng bảo vệ, giúp các thông tin gốc không bị thất thoát hay hao hụt sau khi đi qua các lớp tính toán phức tạp, đồng thời giúp mạng cập nhật gradient trơn tru và ổn định hơn.

Tuy nhiên, nguyên tắc của phép cộng là hai khối dữ liệu phải có cùng kích thước. Do đó, nếu số lượng kênh đầu vào và đầu ra đang khác nhau, hệ thống sẽ dùng thêm một phép tích chập 1×1 trên đường vòng trên để điều chỉnh số lượng kênh cho khớp với nhau rồi mới thực hiện phép cộng.

2.5.4 Kiến trúc phân nhánh Cross Stage Partial (CSP)

Mạng CSPNet (Cross Stage Partial Network) giải quyết vấn đề tính toán dư thừa khi các lớp biến đổi được xếp nối tiếp nhau liên tục. Nguyên lý hoạt động của CSP bao gồm:

- Dữ liệu đầu vào được chia làm hai nhánh song song. Nhánh thứ nhất (bottleneck) đi qua chuỗi các khối biến đổi để học những đặc trưng phức tạp. Nhánh thứ hai (shortcut) chỉ đi qua một phép tích chập 1×1 đơn giản nhằm bảo lưu thông tin nền tảng.
- Hai nhánh xử lý này sau đó được ghép nối và hợp nhất thông qua một lớp tích chập 1×1 cuối cùng.

Cách tiếp cận này mang lại hiệu quả vừa cắt giảm một nửa khối lượng tính toán, vừa tạo ra luồng gradient ngắn hơn để ngăn chặn hiện tượng tiêu biến gradient (gradient vanishing) và tăng khả năng tái sử dụng đặc trưng.

2.5.5 Khối tích hợp C3Ghost

Khối C3Ghost là sự kết hợp hoàn hảo giữa khung kiến trúc phân nhánh CSP (bao gồm ba lớp Conv2D 1×1) và các khối cổ chai GhostBottleneck. Biến thể C3GhostV2 được áp dụng với chuỗi $n = 2$ khối GhostBottleneck ghép nối tiếp. Việc gia tăng số lượng khối giúp mạng trích xuất được các pattern hình ảnh phức tạp hơn mà tham số tăng thêm không đáng kể.

Khối C3Ghost được bố trí chiến lược tại hai vị trí trong phần Neck của mô hình:

- Một khối đảm nhiệm xử lý bản đồ đặc trưng kích thước 26×26 sau khi upsample (cung cấp dữ liệu cho Head 1).
- Một khối đảm nhiệm xử lý bản đồ đặc trưng kích thước 52×52 (cung cấp dữ liệu cho Head 3).

Việc sử dụng mô-đun C3Ghost thay thế cho các lớp Conv2D 3×3 tiêu chuẩn đã giúp cắt giảm đáng kể trọng lượng của mô hình tổng thể, trong khi vẫn duy trì và cải thiện năng lực biểu diễn thông tin không gian nhờ sự kết hợp giữa thiết kế CSP và các kết nối tắt.

2.6 NÉN MÔ HÌNH BẰNG PHƯƠNG PHÁP TUCKER DECOMPOSITION

2.6.1 Nhu cầu nén mô hình cho thiết bị nhúng

Sau khi tích hợp đầu phát hiện thứ ba, cơ chế chú ý CBAM và thay thế các lớp bằng C3Ghost và GSConv, độ chính xác của mô hình đề xuất đã vượt trội hơn đáng kể so với Tiny YOLOv3 gốc. Tuy nhiên, tổng khối lượng tham số vẫn còn cao xấp xỉ 7.5 triệu. Việc triển khai mô hình này lên bo mạch nhúng Raspberry Pi 4 vẫn gặp phải rào cản lớn về tốc độ suy luận do thiếu bộ tăng tốc ma trận chuyên dụng.

Để giải quyết bài toán này, các kỹ thuật nén mô hình được xem xét. Trong số đó, Tucker Decomposition [12], [14] là một phương pháp phân tích hạng thấp dành cho tensor nhiều chiều — được lựa chọn. Ưu điểm cốt lõi của phương pháp này là có thể áp dụng trực tiếp lên bộ trọng số đã được huấn luyện trước, đồng thời có thể tái biểu diễn dưới dạng các lớp tích chập tiêu chuẩn, đảm bảo tính tương thích hoàn toàn với các framework Edge AI như ONNX Runtime.

2.6.2 Phân tích Tensor trọng số Conv2D

Trọng số của một lớp tích chập 2D (Conv2D) được biểu diễn dưới dạng một tensor bốn chiều $\mathbf{W} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$

Trong đó:

- $k \times k$ là kích thước cửa sổ không gian.
- C_{in}, C_{out} : lần lượt là số kênh đầu vào và đầu ra.

Các quan sát thực nghiệm cho thấy các chiều kênh thường có "hạng thực tế" (effective rank) thấp hơn nhiều so với kích thước đầy đủ của chúng, đồng nghĩa với việc các hàng hoặc cột có sự tương quan cao và không gian biểu diễn thực chất chỉ cần một số chiều nhỏ hơn để lưu trữ đầy đủ lượng thông tin.

Dựa trên đặc tính này, phương pháp Partial Tucker Decomposition [12], [13] tiến hành phân rã tensor $\mathbf{W}$ dọc theo hai chiều kênh (mode 2 và mode 3), đồng thời bảo toàn hai chiều không gian $k \times k$:

$$\mathbf{W} \approx \mathbf{G} \times_2 \mathbf{U}_{in} \times_3 \mathbf{U}_{out}, \quad (2.16)$$

Trong đó:

- $\mathbf{G} \in \mathbb{R}^{k \times k \times R_{in} \times R_{out}}$: tensor lõi, với $R_{in} < C_{in}$ và $R_{out} < C_{out}$ là các hạng rank đã được rút gọn
- $\mathbf{U}_{in} \in \mathbb{R}^{C_{in} \times R_{in}}$ và $\mathbf{U}_{out} \in \mathbb{R}^{C_{out} \times R_{out}}$: lần lượt là các ma trận nhân tử phân rã dọc theo chiều kênh đầu vào và ra
- Phép phân rã này được thực hiện thông qua thuật toán Partial HOSVD (từ thư viện TensorLy) nhằm tìm ra các ma trận nhân tử tối ưu nhất theo phương pháp bình phương tối thiểu.

2.6.3 Tái cấu trúc thành ba lớp tích chập nhỏ

Về mặt tính toán thực tế trong mạng học sâu, phép nhân tensor trong công thức trên có thể được ánh xạ tương đương thành cấu trúc ba lớp tích chập xếp nối tiếp nhau:

- Lớp thu hẹp (*prev*): Một lớp $\text{Conv2D}(R_{in}, 1 \times 1)$ không sử dụng bias, thực hiện phép chiếu tuyến tính để nén C_{in} kênh đầu vào xuống R_{in} kênh thông qua ma trận nhân tử $\mathbf{U}_{in}$.
- Lớp lõi (*core*): Một lớp $\text{Conv2D}(R_{out}, k \times k)$ không sử dụng bias, tiến hành trích xuất đặc trưng không gian trên số kênh đã rút gọn dựa vào trọng số của tensor lõi $\mathbf{G}$.
- Lớp khôi phục (*post*): Một lớp $\text{Conv2D}(C_{out}, 1 \times 1)$ dùng ma trận $\mathbf{U}_{out}$ để ánh xạ ngược từ R_{out} kênh trở về kích thước C_{out} kênh như ban đầu.

Sự phân tách này giúp tổng lượng tham số giảm mạnh từ $k^2 \cdot C_{in} \cdot C_{out}$ xuống chỉ còn $(C_{in} \cdot R_{in} + k^2 \cdot R_{in} \cdot R_{out} + R_{out} \cdot C_{out})$, mang lại hiệu quả cực kỳ rõ rệt khi $R_{in} \ll C_{in}$ và $R_{out} \ll C_{out}$.

2.6.4 Xác định Rank và hệ số nén

Các giá trị rank R_{in} và R_{out} của mỗi lớp được tính toán dựa trên một hệ số nén (compression factor) f , được điều chỉnh linh hoạt theo chức năng của lớp đó trong mạng:

$$R_{in} = \lfloor f \times C_{in} \rfloor, \quad R_{out} = \lfloor f \times C_{out} \rfloor, \quad (2.17)$$

Hệ số f càng nhỏ thì tỉ lệ nén càng cao nhưng sai số xấp xỉ cũng tăng theo. Cụ thể, các lớp nông ở đầu Backbone (nơi trích xuất cạnh, góc) được nén nhẹ với $f = 0.70$ để bảo toàn thông tin cơ bản. Ngược lại, các lớp càng sâu với số lượng kênh càng lớn sẽ bị nén mạnh hơn, với f giảm dần xuống 0.55, 0.35 và chạm đáy ở mức 0.20. Quá trình phân rã

chỉ áp dụng lên 6 lớp Conv2D(3×3) nặng nhất thuộc Backbone. Các khối như C3Ghost, GSConv và đầu phát hiện không bị nén để bảo toàn năng lực dự đoán cuối cùng.

CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT

3.1 TỔNG QUAN PIPELINE CẢI TIẾN

3.1.1 Chiến lược cải tiến tổng thể

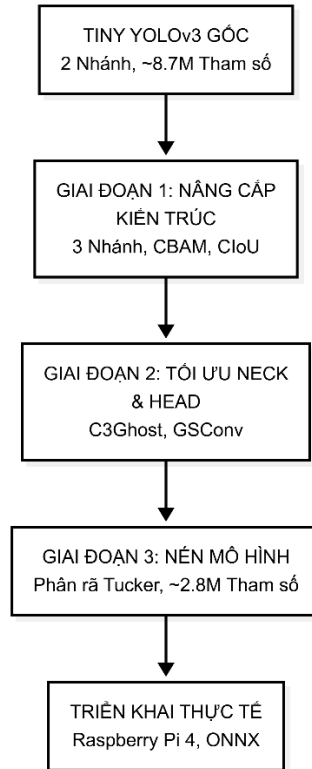
Dựa trên các hạn chế của Tiny YOLOv3 nguyên bản, đề tài đề xuất một quy trình cải tiến bao gồm ba giai đoạn kế thừa chặt chẽ. Mục tiêu của cải tiến là giảm số lượng tham số, chi phí tính toán và khắc phục các điểm yếu về độ chính xác và khả năng nhân diện đa tỷ lệ, nhưng vẫn bảo toàn về ưu thế tốc độ và sự gọn nhẹ của mô hình.

Mỗi giai đoạn cải tiến được thiết kế để giải quyết từng vấn đề cụ thể, tạo tiền đề vững chắc cho các giai đoạn tiếp theo:

- Giai đoạn 1 (Nâng cấp kiến trúc): Tập trung tối đa hóa độ chính xác bằng cách mở rộng khả năng nhận diện và tích hợp cơ chế tập trung đặc trưng (Attention).
- Giai đoạn 2 (Tối ưu hóa Neck và Head): Thay thế các khối tích chập nặng nề bằng các mô-đun tiên tiến (C3Ghost, GSConv) nhằm cắt giảm khối lượng tính toán (FLOPs).
- Giai đoạn 3 (Nén mô hình): Áp dụng kỹ thuật nén hạng thấp để thu gọn kích thước mô hình, đáp ứng nghiêm ngặt các ràng buộc về phần cứng của thiết bị nhúng (Raspberry Pi 4) trước khi triển khai thực tế.

3.1.2 Sơ đồ luồng xử lý ba giai đoạn

Dưới đây là sơ đồ tổng quát mô tả sự dịch chuyển kiến trúc qua từng giai đoạn cải tiến:



Hình 3.1: Pipeline cải tiến của đề tài

3.1.3 Mối liên hệ logic giữa các giai đoạn

Sự phân chia các giai đoạn không phải là sự sắp xếp ngẫu nhiên mà là một chuỗi tiến hóa có tính ràng buộc.

Giai đoạn	Mục tiêu	Thành phần can thiệp	Thành phần bảo toàn
1 – Kiến trúc	Nâng cao mAP	Số lượng Head, Anchor, tích hợp CBAM, hàm Loss	Backbone, toàn bộ trọng số gốc
2 – Tối ưu	Giảm FLOPs	Neck (thay bằng C3Ghost), Head 1 (thay bằng GSConv)	Backbone, Head 2, Head 3, khối CBAM
3 – Nén	Thu nhỏ Model Size	Phân rã 6 lớp Conv2D trong Backbone	Khối Neck, Head, CBAM đã tối ưu

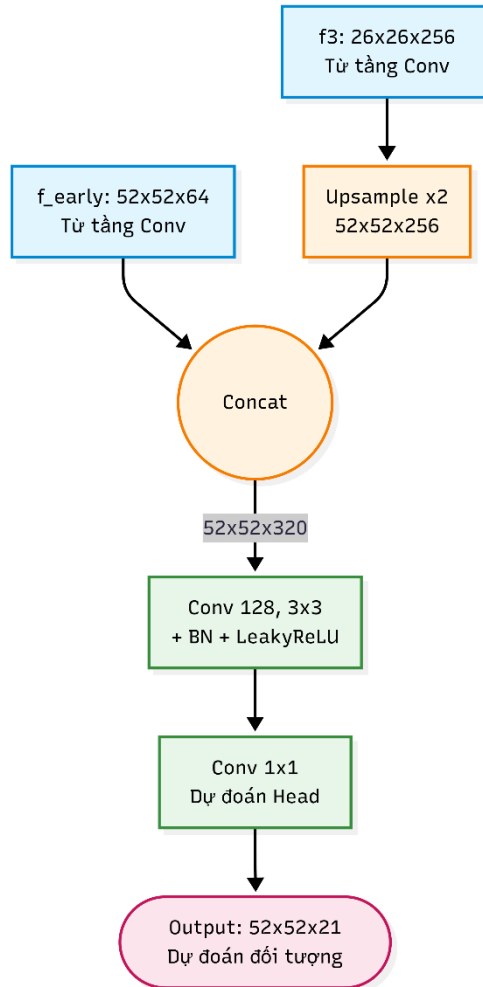
Bảng 3.1: Tóm tắt mục tiêu và thành phần tác động của ba giai đoạn cải tiến

3.2 GIAI ĐOẠN 1 – THÊM DETECTION HEAD THỨ 3 + CBAM

3.2.1 Bổ sung đầu phát hiện thứ ba với tỷ lệ 52×52

Hạn chế lớn nhất của kiến trúc Tiny YOLOv3 nguyên bản là chỉ thiết lập hai đầu phát hiện tại độ phân giải 13×13 và 26×26 . Thiết kế này khiến mô hình mất đi khả năng nhận dạng các đối tượng có kích thước nhỏ, do các đối tượng này chỉ chiếm một lượng pixel rất nhỏ và thông tin chủ yếu nằm ở các tầng đặc trưng có độ phân giải cao.

Để giải quyết triệt để điểm yếu này, đề tài đề xuất bổ sung đầu phát hiện thứ ba (Head 3) tại tỷ lệ 52×52 . Bản đồ đặc trưng tại tỷ lệ này lưu giữ được lượng thông tin không gian chi tiết hơn rất nhiều. Cụ thể, đặc trưng f_3 ($26 \times 26 \times 256$) được trích xuất từ Backbone sẽ được upsample lên kích thước 52×52 , sau đó được ghép nối (concatenate) với bản đồ đặc trưng $f_{early}(52 \times 52 \times 64)$ lấy từ các tầng nông hơn. Sự kết hợp này đảm bảo Head 3 sở hữu một bộ đặc trưng phong phú, bao gồm cả thông tin ngữ nghĩa mức cao từ lớp sâu và thông tin vị trí không gian từ lớp nông.



Hình 3.2: Sơ đồ cấu trúc nhánh phát hiện thứ ba (Head 3) tại độ phân giải 52x52

Việc bổ sung thêm head thứ, tổng số lượng anchor box được mở rộng từ 6 lên 9 hộp, phân bổ đồng đều 3 hộp cho mỗi nhánh. Thay vì sử dụng bộ anchor mặc định của hệ thống YOLOv3 (vốn được tối ưu cho tập dữ liệu COCO 80 lớp), đề tài áp dụng thuật toán phân cụm K-means trực tiếp lên tập dữ liệu huấn luyện riêng. Cách tiếp cận này giúp các anchor bám sát phân phối kích thước thực tế của người đi bộ và phương tiện, đẩy nhanh tốc độ hội tụ và độ chuẩn xác của mô hình.

3.2.2 Tích hợp cơ chế chú ý CBAM

Để nâng cao khả năng phân tách đối tượng khỏi bối cảnh nền phức tạp, mô-đun chú ý CBAM (Convolutional Block Attention Module) được chèn vào ngay sau các luồng đặc trưng f_3 (52×52), f_2 (26×26) và tại nhánh 13×13 trước khi chúng tiến vào Neck/Head.

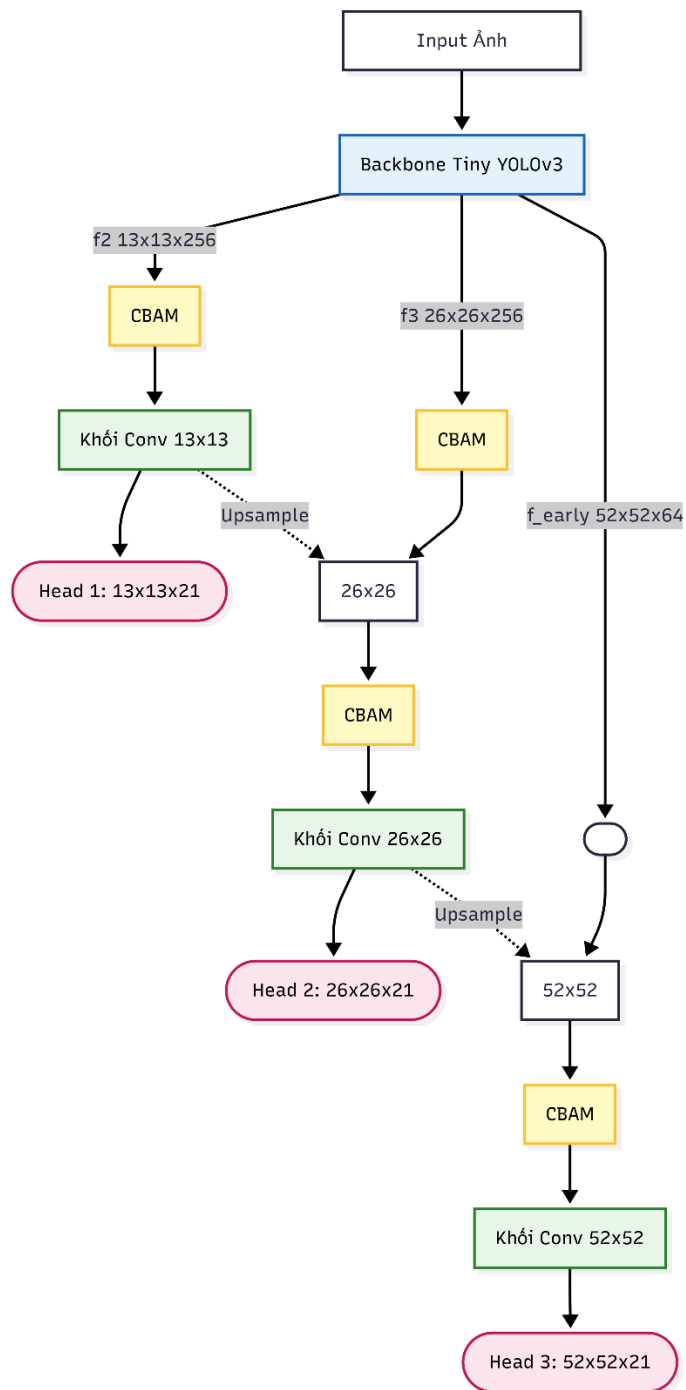
Quy trình tinh chỉnh đặc trưng của khối CBAM diễn ra qua hai giai đoạn:

- Channel Attention: Khai thác mạng MLP dùng chung trọng số kết hợp với các phép gộp Global Average Pooling và Global Max Pooling để đánh giá mức độ quan trọng của từng kênh thông tin, sau đó chuẩn hóa qua hàm Sigmoid.
- Spatial Attention: Tiếp nhận đầu ra từ bước trước, thực hiện gộp dữ liệu theo chiều kênh và áp dụng phép tích chập với cửa sổ 7×7 để tạo ra bản đồ làm nổi bật các khu vực chứa đối tượng trên không gian ảnh

Với việc thiết lập tham số tỷ lệ thu hẹp $r = 8$, hệ thống ba khối CBAM chỉ làm tăng thêm khoảng 19.000 tham số, tương đương chưa tới 0.3% tổng khối lượng tham số của mạng nhưng lại cung cấp khả năng lọc nhiễu xuất sắc, đặc biệt khi đối tượng bị che khuất hoặc hòa lẫn màu sắc với môi trường.

3.2.3 Kiến trúc tổng thể sau giai đoạn 1

Sự kết hợp giữa 3 nhánh phát hiện và các khối CBAM tạo ra một luồng xử lý thông tin liền mạch. Sơ đồ kiến trúc tổng thể sau giai đoạn 1:



Hình 3.3: Sơ đồ kiến trúc tổng thể sau giai đoạn 1

3.3 GIAI ĐOẠN 2 – GSConv + C3GHOST V2 TRONG NECK VÀ HEAD

3.3.1 Động lực cải tiến

Tại Giai đoạn 1, phần Neck của mô hình vẫn sử dụng lớp tích chập *Conv* 3×3 tiêu chuẩn. Với số lượng kênh lớn (256, 512), lớp này trở thành nút thắt cổ chai về mặt tính toán và thiếu đi cơ chế tái sử dụng đặc trưng. Do đó, Giai đoạn 2 thực hiện thay thế chiến lược các khối này bằng C3Ghost cho phần Neck cần tổng hợp nhiều nguồn và GSConv cho phần Head cần lớp biến đổi đơn lẻ, mạnh mẽ.

3.3.2 Tích hợp C3Ghost trong phần Neck

Khối C3Ghost được bố trí tại hai vị trí trọng yếu nhằm xử lý dữ liệu sau ghép nối:

- Khối 1 (Tỷ lệ 26x26): Xử lý tensor $26 \times 26 \times 512$ để xuất ra $26 \times 26 \times 256$ cung cấp cho Head 2.
- Khối 2 (Tỷ lệ 52x52): Xử lý tensor $52 \times 52 \times 320$ để xuất ra $52 \times 52 \times 128$ cung cấp cho Head 3.

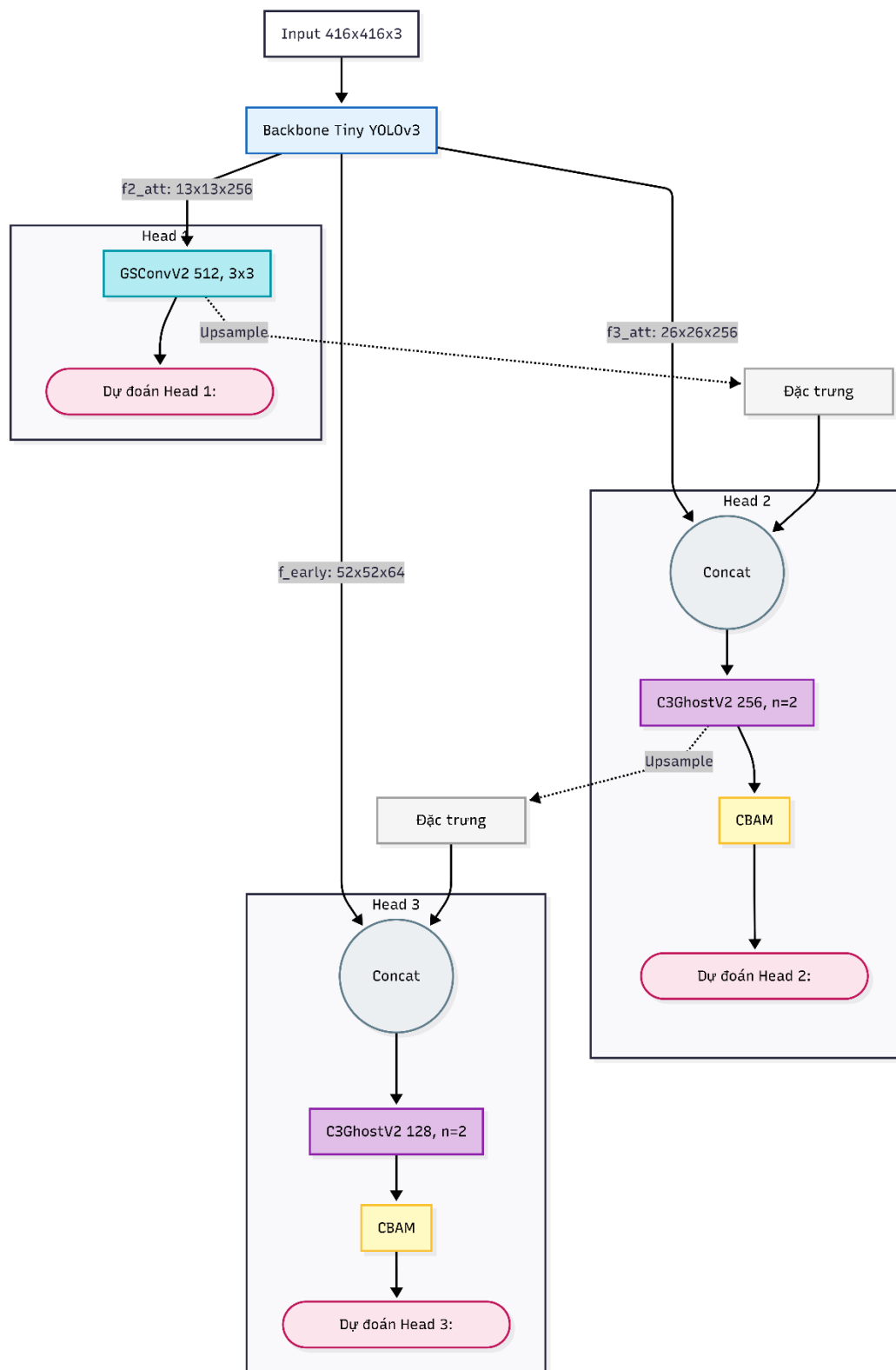
Cả hai khối đều được thiết lập với $n = 2$ (sử dụng 2 khối GhostBottleneck nối tiếp) nhằm gia tăng độ sâu phi tuyến, đảm bảo mô hình học được các biểu diễn phức tạp khi dung hợp đặc trưng từ nhiều nhánh.

3.3.3 Tích hợp GSConvV2 tại Head 1

Nhánh phát hiện đầu tiên (Head 1 tại 13×13) là nơi đòi hỏi khả năng xử lý số lượng kênh lớn nhất (512 kênh). Lớp Conv 3×3 nguyên bản tại đây được thay thế hoàn toàn bằng mô-đun GSConvV2. GSConvV2 áp dụng tích chập tiêu chuẩn cho nửa số kênh và tích chập chiều sâu cho nửa còn lại, sau đó dùng kỹ thuật Channel Shuffle để đan xen luồng thông tin. Việc thay thế này giúp cắt giảm tới xấp xỉ 45% lượng FLOPs tại riêng lớp này mà vẫn duy trì khả năng biểu diễn không gian và ngữ nghĩa.

3.3.4 Kiến trúc tổng thể mạng nơ-ron sau Cải tiến 2

Sự kết hợp giữa Giai đoạn 1 (3 nhánh + CBAM) và Giai đoạn 2 (C3Ghost + GSConv) tạo ra một bộ khung hoàn chỉnh, cân bằng xuất sắc giữa độ chính xác và chi phí phân cứng.



Hình 3.4: Kiến trúc tổng thể tích hợp C3GhostV2 và GSConvV2 tại Giai đoạn 2

3.4 GIAI ĐOẠN 3 - NÉN MODEL BẰNG PHƯƠNG PHÁP TUCKER DECOMPOSITION

3.4.1 Lựa chọn lớp mục tiêu tối ưu

Sau các cải tiến ở 2 giai đoạn trên, mô hình giảm xuống khoảng 7.5 triệu tham số cùng độ chính xác được cải thiện rõ rệt. Tuy nhiên, để có thể đáp ứng được các ràng buộc khắt khe về tài nguyên tính toán và tốc độ suy luận thực tế trên bo mạch Raspberry Pi 4, Giai đoạn 3 áp dụng kỹ thuật Phân rã Tucker một phần (Partial Tucker Decomposition). Đề tài quyết định chỉ nhắm mục tiêu nén vào 6 lớp tích chập Conv2D bên trong khối Backbone, dựa trên 3 lý do cốt lõi sau:

- Sáu lớp Conv 3×3 là các lớp tích chập tiêu chuẩn chứa lượng tham số lớn nhất, từ đó ta có thể áp dụng thuật toán phân rã cho các lớp này
- Các khối chức năng đã thêm ở phần Neck và Head (như các khối GhostConv và GSConv) vốn đã mang kiến trúc nhỏ, tối giản từ giai đoạn 2, do đó không còn nhiều lượng tham số thừa để có thể tiếp tục cắt giảm.
- Việc nén các khối CBAM và Detection Heads sẽ tác động trực tiếp đến khả năng dự đoán cuối cùng, gây ra mức suy giảm mAP vượt quá giới hạn cho phép.

3.4.2 Thiết lập hệ số nén (Compression Factor) cho từng lớp

Kích thước hạng (Rank) sau phân rã của mỗi lớp được xác định dựa trên công thức

$$R = \lfloor f \times C \rfloor, \quad (3.1)$$

Trong đó f là hệ số nén thực nghiệm. Quy tắc thiết lập hệ số f tuân theo nguyên lý:

- Các lớp nông (trích xuất đặc trưng vật lý cơ bản như cạnh, góc) cần được nén nhẹ ứng với hệ số nén f lớn để tránh làm mất mát thông tin nền tảng.
- Các lớp sâu nơi thông tin mang tính trừu tượng cao có sự dư thừa lớn nên sẽ bị nén mạnh ứng với hệ số nén f nhỏ

Tên lớp (Layer)	Chuyển đổi kênh ($C_{in} \rightarrow C_{out}$)	Hệ số nén f	Hạng rút gọn (R_{in}/R_{out})	Căn cứ thiết lập
--------------------	--	------------------	--------------------------------------	------------------

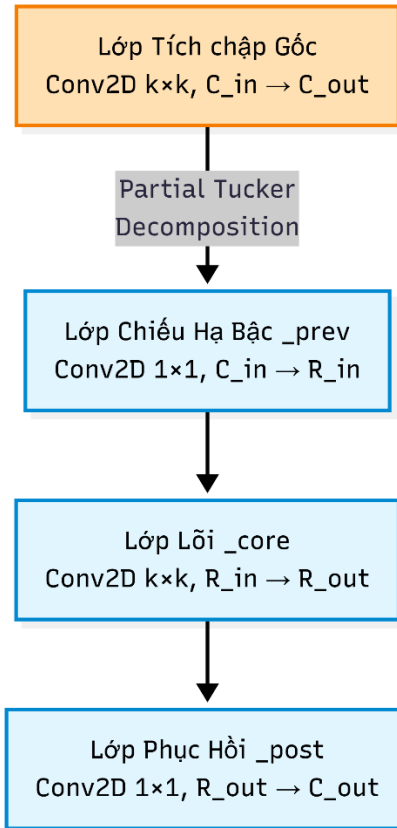
conv2d_1	16 → 32	0.70	11/22	Lớp đầu tiên, xử lý đặc trưng cơ sở (Nén nhẹ)
conv2d_2	32 → 64	0.70	22/44	Lớp đầu tiên, xử lý đặc trưng cơ sở (Nén nhẹ)
conv2d_3	64 → 128	0.55	35/70	Cung cấp luồng f_{early} cho nhánh 52x52 (Nén vừa)
conv2d_4	128 → 256	0.55	70/140	Cung cấp luồng f_3 cho nhánh 26×26 (Nén vừa)
conv2d_5	256 → 512	0.35	89/179	Lớp sâu, tính dư thừa cao (Nén mạnh)
conv2d_6	512 → 1024	0.20	102/204	Nút thắt lớn nhất của mạng (Nén cực mạnh)

Bảng 3.2: Thiết lập hệ số nén Tucker Decomposition cho các lớp Conv2D

3.4.3 Tái cấu trúc lớp Conv2D thành khối ba lớp nối tiếp

Sau khi xác định được các hạng mục tiêu, mỗi lớp Conv2D ($k \times k$) trong Backbone được tách thành một chuỗi ba lớp tích chập nhỏ thông qua thuật toán HOSVD (cung cấp bởi thư viện TensorLy). Sự phân tách này giúp tiết kiệm lượng lớn tham số do $R_{in} \ll C_{in}$ và $R_{out} \ll C_{out}$.

- Lớp *_prep* (Conv 1×1): Chiếu không gian C_{in} kênh đầu vào xuống không gian hạng thấp R_{in} kênh thông qua ma trận nhân tử $\mathbf{U}_{in}$.
- Lớp *_core* (Conv $k \times k$): Trích xuất đặc trưng không gian trực tiếp trên không gian đã nén, sử dụng trọng số từ tensor lõi G .
- Lớp *_post* (Conv 1×1): Phục hồi (ánh xạ ngược) từ R_{out} kênh trở lại kích thước C_{out} kênh nguyên bản bằng ma trận $\mathbf{U}_{out}$.

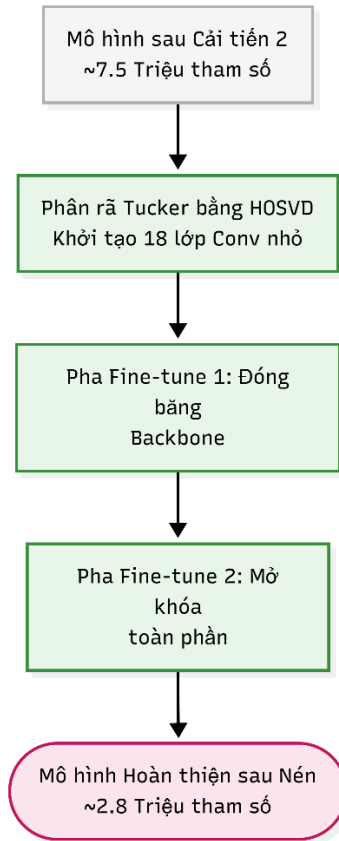


Hình 3.5: Sự biến đổi kiến trúc của một lớp Conv2D qua Phân rã Tucker

3.4.4 Quy trình tinh chỉnh (Fine-tuning) hai pha sau nén

Vì thuật toán HOSVD bản chất là một phép xấp xỉ toán học, bộ trọng số sinh ra của 3 lớp mới chưa hoàn toàn gắn kết với hàm mất mát (loss function) của hệ thống nhận diện. Để khôi phục độ chính xác, mô hình phải trải qua quy trình tinh chỉnh hai pha:

- Pha 1 (Freeze): Đóng băng toàn bộ trọng số Backbone vừa nén, chỉ mở khóa huấn luyện khối Neck, Head và CBAM với hệ số học (learning rate) tương đối cao. Việc này ép các khối dự đoán phải thích nghi nhanh với luồng đặc trưng mới mà không làm hỏng kết quả khởi tạo của HOSVD.
- Pha 2 (Unfreeze): Mở khóa toàn phần mạng nơ-ron và hạ thấp hệ số học để tất cả tham số cùng tiến về một điểm hội tụ chung, thu hẹp tối đa sự chênh lệch mAP so với trước khi nén.



Hình 3.6: Lưu đồ quy trình phân rã và tinh chỉnh trọng số mạng nơ-ron

3.4.5 Tổng hợp kết quả và nhận xét

Sự kết hợp giữa Partial Tucker Decomposition và chiến lược tinh chỉnh hai pha đã mang lại thành công lớn. Khối lượng tham số của mô hình được cắt giảm tới 63%, từ 7.5 triệu xuống chỉ còn xấp xỉ 2.8 triệu tham số. Sự hao hụt này chủ yếu đến từ các lớp sâu nhất như *conv2d_5* và *conv2d_6*, phản ánh đúng lượng thông tin dư thừa không lồ ở chặng cuối của mạng trích xuất. Chỉ số $mAP@0.5$ ghi nhận độ suy hao cực kỳ nhỏ gọn, chứng minh hiệu quả tuyệt vời của phương pháp Fine-tuning trong việc bù đắp các sai số xấp xỉ.

CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ

4.1 DATASET

4.1.1 Nguồn dữ liệu

Tập dữ liệu sử dụng trong nghiên cứu được xây dựng từ hai bộ dữ liệu chuẩn trong lĩnh vực nhận dạng đối tượng trên xe tự lái: Caltech Pedestrian Dataset [1] và KITTI Vision Benchmark Suite [2]. Việc kết hợp hai nguồn dữ liệu nhằm tăng tính đa dạng về điều kiện thu thập, góc nhìn camera và mật độ đối tượng, qua đó giúp mô hình học được đặc trưng tổng quát hơn. Mô hình được huấn luyện để nhận dạng hai lớp đối tượng chính là người đi bộ (person, class 0) và ô tô (car, class 1).

Caltech Pedestrian Dataset được thu thập tại khu vực đô thị Los Angeles, Hoa Kỳ, bằng cách gắn camera lên xe ô tô di chuyển trên đường phố. Tập dữ liệu này được thiết kế chuyên biệt nhằm đánh giá các phương pháp phát hiện người đi bộ trong điều kiện thực tế, bao gồm nhiều trường hợp khó như người bị che khuất một phần (partial occlusion), người ở khoảng cách xa với kích thước nhỏ trong khung hình, và sự thay đổi về điều kiện ánh sáng theo từng thời điểm trong ngày. Mỗi ảnh có độ phân giải 640×480 pixel và được đặt tên theo cấu trúc setXX_VXXX_XXXX.png, phản ánh số hiệu tập dữ liệu (set) và chuỗi video (video sequence) tương ứng.

KITTI Vision Benchmark Suite được thu thập tại thành phố Karlsruhe, Đức, sử dụng hệ thống camera stereo gắn trên xe, phục vụ nhiều tác vụ trong lĩnh vực xe tự lái như phát hiện đối tượng 2D/3D, ước lượng độ sâu và phân đoạn ngữ nghĩa. Ảnh trong tập dữ liệu này có độ phân giải 1242×375 pixel với tỷ lệ khung hình rộng, phản ánh trường nhìn của camera stereo đặt ở mũi xe. KITTI chứa đa dạng phương tiện trong các cảnh ngoại ô và xa lộ với mật độ ô tô cao, hỗ trợ tốt cho lớp car so với Caltech. Tên file ảnh được đánh số thứ tự liên tiếp dạng 000000.png, 000001.png, ...

4.1.2 Thống kê tập dữ liệu

Toàn bộ dữ liệu sau khi tổng hợp gồm 20.261 ảnh, trong đó Caltech chiếm 12.780 ảnh (63,1%) và KITTI chiếm 7.481 ảnh (36,9%). Tập dữ liệu được phân chia theo tỷ lệ 80% huấn luyện / 20% kiểm thử, cho kết quả 16.209 ảnh dùng để huấn luyện và 4.052 ảnh dùng để kiểm thử.

	Train	Validation	Test	Tổng
Tổng số ảnh	16,209	4,052	3,138	23,399
BBox person	22,951	5,940	4,515	33,406
BBox car	75,944	18,611	20,496	115,051
Tổng BBox	98,895	24,551	25,011	148,457

Bảng 4.1: Trình bày thống kê chi tiết số lượng ảnh và bounding box theo từng tập.

4.1.3 Phân tích sự mất cân bằng dữ liệu

Qua thống kê cho thấy tập dữ liệu tồn tại sự mất cân bằng đáng kể giữa hai lớp đối tượng. Trên tập huấn luyện gốc, lớp car có 75.944 bounding box trong khi lớp person chỉ có 22.951 bounding box, tương đương tỷ lệ 3,3:1 nghiêng về phía ô tô. Nguyên nhân chủ yếu đến từ đặc thù của KITTI — bộ dữ liệu này được thiết kế cho bài toán xe tự lái nên mật độ phương tiện trên mỗi khung hình rất cao, trong khi Caltech tuy nhiều ảnh hơn nhưng lại tập trung vào người đi bộ với số lượng instance mỗi ảnh thấp hơn.

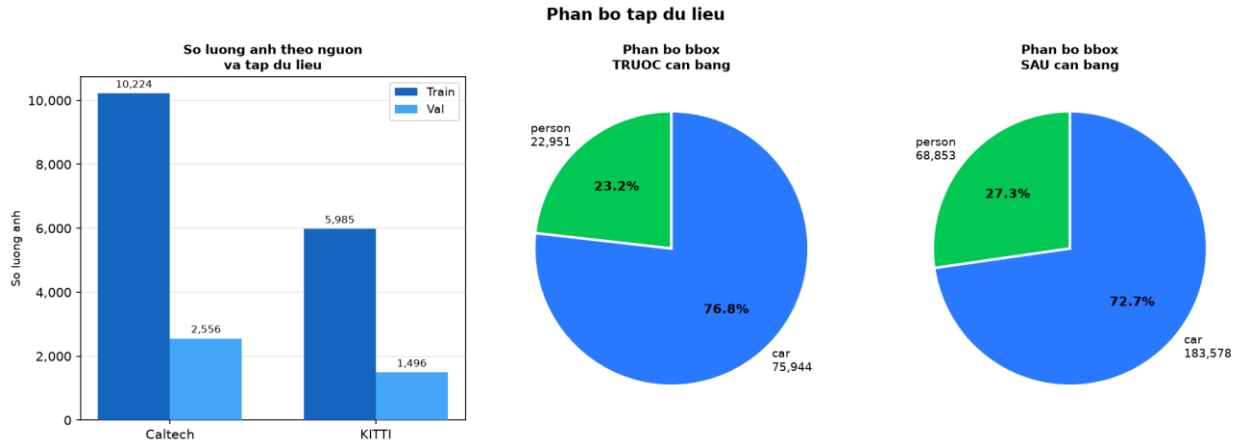
Sự mất cân bằng này nếu không được xử lý sẽ khiến mô hình bị thiên lệch (bias) về phía lớp chiếm đa số, dẫn đến chỉ số recall thấp đối với người đi bộ — vốn là đối tượng cần ưu tiên trong các ứng dụng an toàn giao thông.

4.1.4 Cân bằng dữ liệu

Để giảm thiểu ảnh hưởng của sự mất cân bằng, chiến lược oversampling được áp dụng: các ảnh có chứa ít nhất một bounding box thuộc lớp person được nhân đôi trong tập huấn luyện (duplicate factor = 2). Kết quả tạo ra file train_balanced.txt với 39.523 ảnh và 252.431 bounding box tổng cộng, trong đó tỷ lệ car/person giảm từ 3,3:1 xuống còn 2,7:1.

	Gốc	Cân bằng
Số ảnh	16.209	39.523
BBox person	22.951	68.853
BBox car	75.944	183.578
Tỉ lệ	3,3:1	2,7:1

Bảng 4.2: So sánh phân bố dữ liệu trước và sau khi cân bằng.



Hình 4.1: Biểu đồ phân bố tập dữ liệu

File `train_balanced.txt` này được sử dụng làm đầu vào chính cho quá trình huấn luyện, trong khi tập validation `val.txt` giữ nguyên phân bố gốc để đảm bảo việc đánh giá mô hình phản ánh đúng điều kiện th 4.1.5 Định dạng annotation và tiền xử lý.

File `train_balanced.txt` này giúp mô hình tính toán lại chỉ số anchor box để phù hợp dự đoán cho cả người và xe. Với người thì hộp dự đoán sẽ cao và hẹp, còn đối với xe thì hộp dự đoán sẽ dài và thấp nên việc cân bằng lại số lượng người trên tập dữ liệu cũng làm giảm tỷ lệ chênh lệch anchor box của xe đối với người giúp tăng tỷ lệ học lớp người của mô hình.

Cả hai nguồn dữ liệu đều sử dụng định dạng annotation YOLO chuẩn (Darknet format), trong đó mỗi bounding box được biểu diễn bằng năm giá trị `class_id cx cy w h` với tọa độ tâm và kích thước được chuẩn hóa về đoạn $[0, 1]$. Để tương thích với framework `keras-yolo3`, các annotation này được chuyển đổi sang tọa độ pixel tuyệt đối theo cú pháp $x_{min}, y_{min}, x_{max}, y_{max}, class_id$.

Do hai nguồn dữ liệu có độ phân giải khác nhau (640×480 và 1242×375):



Hình 4.2: So sánh hai nguồn dữ liệu

Trong giai đoạn tiền xử lý dữ liệu, ảnh đầu vào được chuẩn hóa bằng kỹ thuật Letterbox Resize, trong đó ảnh được thu phóng về kích thước chuẩn 416×416 pixel nhưng vẫn giữ nguyên tỷ lệ khung hình. Phần không gian còn thiếu được bổ sung bằng viền màu xám có giá trị (128, 128, 128). Phương pháp này giúp hạn chế hiện tượng méo hình học thường gặp khi sử dụng resize thông thường, đồng thời đảm bảo các bounding box không bị biến dạng trong quá trình huấn luyện.

Bên cạnh đó, thay vì sử dụng bộ anchor mặc định của COCO, các anchor được xây dựng bằng thuật toán K-Means Clustering trên tập hợp toàn bộ bounding box của bộ dữ liệu kết hợp Caltech Pedestrian và KITTI. Cách tiếp cận này giúp tạo ra các anchor có kích thước phù hợp hơn với phân bố thực tế của các đối tượng như người đi bộ và phương tiện giao thông, từ đó nâng cao hiệu quả phát hiện đối tượng trong môi trường giao thông và đô thị.

4.2 THIẾT LẬP HUẤN LUYỆN

4.2.1 Cấu hình phần cứng và phần mềm

Toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên máy tính cá nhân với cấu hình phần cứng GPU NVIDIA RTX 1650 Ti và Intel Core i5-11400H @ 2.70 GHz (6 nhân / 12 luồng) với 4 GB VRAM được sử dụng làm thiết bị tăng tốc tính toán chính.

Framework chính được sử dụng là TensorFlow-GPU 2.10.0 kết hợp CUDA 11.2 và cuDNN 8.1. Đây là phiên bản TensorFlow cuối cùng hỗ trợ GPU native trên Windows - từ

phiên bản 2.11 trở đi, TensorFlow yêu cầu WSL2 để sử dụng GPU trên hệ điều hành Windows, do đó phiên bản 2.10.0 được chọn nhằm đơn giản hóa môi trường triển khai. Môi trường được quản lý bằng Conda Bảng 4.4 liệt kê các thư viện phần mềm chính.

Thư viện	Phiên bản	Vai trò
Python	3.9	Ngôn ngữ lập trình
TensorFlow-GPU	2.10.0	Framework huấn luyện và suy luận
Keras	2.10.0	API xây dựng mô hình
CUDA / cuDNN	11.2/8.1	Kích hoạt gpu và chạy
NumPy	1.24.3	Tính toán ma trận
OpenCV	4.8.1	Xử lý ảnh và video
TensorLy	0.9.0	Lượng tử Tucker nén mô hình
TensorBoard	2.10.0	Theo dõi quá trình huấn luyện

Bảng 4.3: Các thư viện phần mềm chính.

4.2.2 Thiết lập huấn luyện

Mô hình và dữ liệu đầu vào: Mô hình đề xuất là Tiny YOLOv3 cải tiến với ba đầu ra phát hiện (3-head), tích hợp khối GSConv, cấu trúc C3Ghost-v2 và cơ chế chú ý CBAM. Toàn bộ ảnh đầu vào được resize về 416×416 pixel thông qua kỹ thuật letterbox giữ nguyên tỷ lệ khung hình. Mô hình sử dụng 9 anchor box được tính lại bằng K-Means clustering trên tập dữ liệu Caltech + KITTI sau khi cân bằng, phân bổ đều cho ba tỷ lệ phát hiện: đầu ra 13×13 phụ trách đối tượng lớn, 26×26 phụ trách đối tượng vừa, và 52×52 phụ trách đối tượng nhỏ.

Dữ liệu huấn luyện sử dụng file train_balanced.txt (39.523 ảnh sau oversampling) kết hợp với kỹ thuật Mosaic augmentation — mỗi sample có xác suất 50% được ghép từ bốn ảnh khác nhau thành một ảnh tổng hợp, tối đa 80 bounding box mỗi ảnh. Mosaic augmentation giúp mô hình học được ngữ cảnh đa dạng hơn và tăng cường khả năng phát hiện đối tượng nhỏ.

Hàm loss: Thay vì sử dụng MSE để hồi quy tọa độ bounding box như Tiny YOLOv3 gốc, mô hình áp dụng CIoU Loss (Complete Intersection over Union) giúp tối ưu đồng thời ba yếu tố: khoảng cách tâm, tỷ lệ khung hình và diện tích overlap. Phân loại đối tượng sử dụng Focal Loss với hệ số $\gamma = 2,0$ nhằm tập trung học vào các mẫu khó phân loại, kết hợp Label Smoothing $\varepsilon = 0,05$ để giảm hiện tượng overconfidence.

Chiến lược huấn luyện hai giai đoạn: Mô hình được huấn luyện theo phương pháp transfer learning, khởi đầu từ trọng số pretrained của phiên bản v2 (GSConv + CIoU, không có Mosaic).

Giai đoạn 1 — Warmup với backbone đóng băng: 20 lớp đầu của backbone được giữ nguyên trọng số, chỉ các lớp detection head được cập nhật. Mục đích là để phần đầu ra thích nghi nhanh với Mosaic augmentation trước khi fine-tune toàn mạng.

Giai đoạn 2 — Fine-tune toàn bộ mạng: Toàn bộ các lớp được mở khóa, learning rate được điều chỉnh theo lịch Cosine Annealing với warmup — tăng tuyến tính trong 3 epoch đầu rồi giảm dần theo hàm cosine đến 1% learning rate ban đầu. Chiến lược này giúp tránh bước nhảy learning rate đột ngột khi unfreeze backbone, đồng thời hỗ trợ hội tụ mượt mà hơn so với ReduceLROnPlateau.

Tham số	Giai đoạn 1	Giai đoạn 2
Kích thước ban đầu	416x416	416x416
Batch size	8	4
learning	5×10^{-4}	5×10^{-5}
Số epoch tối đa	5	50
Optimizer	Adam	Adam
Backbone	Đóng băng	Mở khóa toàn bộ
Lịch LR	Cố định	Cosine Annealing (warmup 3 epoch)
Mosaic augmentation	50%	50%
Ignore threshold	0.5	0.5
Focal γ	2	2
Label smoothing ϵ	0.05	0.05
EarlyStopping	-	15 epoch

Bảng 4.4: Thiết lập các thông số train

Quá trình huấn luyện được theo dõi qua TensorBoard, trọng số tốt nhất theo validation loss được lưu tự động bằng ModelCheckpoint sau mỗi epoch.

4.2.3 Thiết lập nén mô hình Tucker

Sau khi huấn luyện mô hình full-rank hoàn tất, quá trình nén Tucker được thực hiện qua hai bước nhằm giảm kích thước mô hình trong khi duy trì độ chính xác.

Bước 1 — Nén kiến trúc (Tucker Decomposition):

Các lớp tích chập 3×3 trong backbone và detection head được thay thế bằng chuỗi ba lớp tích chập nhỏ hơn thông qua phân rã tensor Tucker-2 (phân rã theo chiều input channel và output channel). Tỷ lệ nén được thiết lập khác nhau cho từng lớp tùy theo mức độ nhạy cảm với độ chính xác, cụ thể như trong Bảng 4.6. Các lớp CBAM và lớp FPN concatenation (conv2d_11) không được nén do CBAM không chứa lớp tích chập 3×3 phù hợp cho Tucker, còn conv2d_11 rất nhạy cảm về độ chính xác.

Bảng so sánh các hệ số nén khác nhau:

	conv2d_ 1	conv2d_ 2	conv2d_ 3	conv2d_ 4	conv2d_ 5	conv2d_ 6
Config 1	0.7	0.7	0.6	0.6	0.1	0.15
Config 2	0.7	0.6	0.6	0.5	0.1	0.15
Config 3	0.7	0.65	0.65	0.55	0.1	0.1

Bảng 4.5: Bảng các hệ số nén của ba config khác nhau

Bảng so sánh kết quả các config nén:

	Params	GFLOPs	mAP@0.5 (valid)	mAP@0.5 (test)
Config 1	2.782M	1.23	81.89%	75.52%
Config 2	2.710M	1.19	78.58%	72.46%
Config 3	2.552M	1.19	75.25%	70.53%

Bảng 4.6: Kết quả của 3 config nén khác nhau

Từ bảng trên, ta thấy config 1 có số lượng params và FLOPs phù hợp với các bo nhúng và chỉ số mAP@0.5 giữ ổn định nhất trong 3 config. Vì vậy, hệ số tối ưu nhất để chọn trong đề tài này là config 1.

Lớp	Chuyển đổi kênh ($C_{in} \rightarrow C_{out}$)	Tỷ lệ
conv2d_1	16 $\rightarrow$ 32	0.7
conv2d_2	32 $\rightarrow$ 64	0.7
conv2d_3	64 $\rightarrow$ 128	0.6
conv2d_4	128 $\rightarrow$ 256	0.6
conv2d_5	256 $\rightarrow$ 512	0.10
conv2d_6	512 $\rightarrow$ 1024	0.15
conv2d_14	192 $\rightarrow$ 128	0.7
conv2d_11 (FBN)	-	Bỏ qua
CBAM	-	Bỏ qua

Bảng 4.7: Tỷ lệ nén Tucker theo từng lớp

Bước 2 — Fine-tune sau nén:

Sau khi tái cấu trúc kiến trúc, mô hình được fine-tune lại để khôi phục độ chính xác bị mất. Quá trình này cũng thực hiện theo hai giai đoạn với backbone đóng băng trước, sau đó mở khóa toàn bộ. Bảng 4.7 trình bày siêu tham số fine-tune Tucker.

Tham số	Giai đoạn 1	Giai đoạn 2
Batch size	8	8
Learning rate	1×10^{-4}	1×10^{-4}
Số epoch	10	40
Optimizer	adam	adam
Callbacks	ReduceLROnPlateau, EarlyStoppin	ReduceLROnPlateau, EarlyStoppin

Bảng 4.8: Siêu tham số fine-tune sau nén Tucker

4.3 KẾT QUẢ THỰC NGHIỆM

4.3.1 Kết quả đánh giá độ chính xác

Mô hình đề xuất được đánh giá trên tập validation gồm 4.052 ảnh (2.556 ảnh Caltech và 1.496 ảnh KITTI) với ngưỡng IoU = 0,5 theo chuẩn Pascal VOC. Bảng 4.8 trình bày kết quả chi tiết theo từng lớp đối tượng.

```
--- Inference Time (on 4052 real images) ---
Mean latency : 43.15 ms/img
--- Inference Time (on 4052 real images) ---
Mean latency : 43.15 ms/img
Mean latency : 43.15 ms/img
Median (P50) : 41.29 ms/img
P95          : 55.18 ms/img
FPS           : 23.2

[person]
GT=5940 Predictions=9613
Best conf threshold : 0.466
TP=4486 FP=743 FN=1454
Precision          : 85.79%
Recall             : 75.52%
F1                 : 80.33%
AP@0.5            : 81.36%

[car]
GT=18611 Predictions=29616
Best conf threshold : 0.493
TP=14565 FP=2222 FN=4046
Precision          : 86.76%
Recall             : 78.26%
F1                 : 82.29%
AP@0.5            : 84.59%

-----
mAP@0.5 : 82.97%
```

Hình 4.3: Kết quả sau khi test bằng CPU

Chỉ số	person	car	Trung bình
AP@0.5 (valid)	81.36%	84.59%	82.97%
AP@0.5 (test)	72.18%	77.25%	74.25
Precision	85.79%	86.76%	86.28%
recall	75.52%	78.26%	76.89%
F1-score	80.33%	82.29%	81.31%
conf tối ưu	0.466	0.493	-

Bảng 4.9: Kết quả đánh giá mô hình đề xuất trên tập validation

```

Model : D:\DATN\yolov3_keras\logs\tiny_3head_gsconv_ciou_v3\best_stage2.weights.h5
Params : 7,495,749
GFLOPs : 2.29 (input 416x416)
Device : GPU
Val set : val.txt (4052 images)
Total eval time : 104.1s
--- Inference Time (on 4052 real images) ---
Mean latency : 6.63 ms/img
Median (P50) : 6.25 ms/img
P95 : 8.81 ms/img
FPS : 150.9

[person]
GT=5940 Predictions=9613
Best conf threshold : 0.466
TP=4486 FP=743 FN=1454
Precision : 85.79%
Recall : 75.52%
F1 : 80.33%
AP@0.5 : 81.36%

[car]
GT=18611 Predictions=29616
Best conf threshold : 0.493
TP=14565 FP=2222 FN=4046
Precision : 86.76%
Recall : 78.26%
F1 : 82.29%
AP@0.5 : 84.59%

mAP@0.5 : 82.97%

```

Hình 4.4: Kết quả sau khi test bằng GPU

Kết quả đánh giá khi chạy bằng GPU tương tự như cpu nhưng có tốc độ fps và inference time tốt hơn so với khi chạy bằng CPU.

Mô hình đạt $mAP@0.5 = 82,97\%$ trên toàn bộ tập validation. Lớp car đạt AP cao hơn (84,59%) so với lớp person (81,36%). Sự chênh lệch này xuất phát từ đặc thù của Caltech Pedestrian Dataset — tập dữ liệu này chứa nhiều trường hợp người đi bộ bị che khuất một phần, kích thước nhỏ do ở xa camera, và điều kiện ánh sáng thay đổi. Chỉ số Precision của cả hai lớp đều vượt 85%, cho thấy mô hình có tỷ lệ phát hiện nhầm thấp. Bảng 4.8 trình bày thống kê chi tiết số lượng TP, FP và FN.

	Tổng GT	TP	FP	FN
person	5.940	4.486	743	1.454
car	18.611	14.565	2.222	4.046

Bảng 4.10: Thống kê TP / FP / FN trên tập validation

4.3.2 Độ phức tạp và tốc độ suy luận

Mô hình đề xuất có tổng số tham số 7.495.749 và độ phức tạp tính toán 4,58 GFLOPs với kích thước đầu vào 416×416 . Đây là mức phù hợp cho các thiết bị nhúng có tài nguyên tính toán hạn chế.

Tốc độ suy luận được đo bằng cách chạy 200 lần suy luận trên ảnh 416×416 sau 20 lần warmup để loại bỏ overhead khởi tạo. Kết quả được trình bày trong Bảng 4.10.

Thông số	Giá trị
Số tham số	7.495.749
GFLOPs (416×416)	4,58
Kích thước file weights	29MB
Mean latency	43.15ms / ảnh
Fps(CPU)	23.2
Mean latency	6.63 ms / ảnh
Fps(GPU)	150.9

Bảng 4.11: Thông số mô hình và tốc độ suy luận

4.3.3 Kết quả sau nén Tucker

```

Model : D:\DATN\yolov3_keras\tucker_output\tucker_gsconv_v2_best_body.weights.h5
Params : 2,782,177
Size : 10.9 MB
GFLOPs : 1.23 (input 416x416)
Device : GPU
Val set : val.txt (4052 images)
Total eval time : 99.7s
--- Inference Time (on 4052 real images) ---
Mean latency : 7.07 ms/img
Median (P50) : 6.69 ms/img
P95 : 9.52 ms/img
FPS : 141.5

[person]
GT=5940 Predictions=9910
Best conf threshold : 0.445
TP=4428 FP=921 FN=1512
Precision : 82.78%
Recall : 74.55%
F1 : 78.45%
AP@0.5 : 79.77%

[car]
GT=18611 Predictions=30068
Best conf threshold : 0.479
TP=14528 FP=2332 FN=4083
Precision : 86.17%
Recall : 78.06%
F1 : 81.91%
AP@0.5 : 84.01%

mAP@0.5 : 81.89%

```

Hình 4.5: Kết quả sau khi test bằng GPU của mô hình sau nén

Bảng 4.10 trình bày kết quả so sánh giữa mô hình full-rank và mô hình sau khi áp dụng Tucker Decomposition, được đánh giá trên cùng tập validation 4.052 ảnh với GPU NVIDIA GeForce GTX 1650.

Chỉ số	Full-rank	Tucker	Thay đổi
Số tham số	7.495.749	2.782.177	-62,8%
Kích thước model	29MB	10.9MB	-62.4%
GFLOPs (416×416)	4,58	2,46	-46,3%
mAP@0.5	82,97%	81,89%	-1,08%
AP person	81,36%	79,77%	-1,59%
FPS(GPUGTX 1650)	150.9	141.5	-6.3%
Mean Latency	6.63 ms / ảnh	7.07 ms / ảnh	6.63%

Bảng 4.12: So sánh mô hình Full-rank và Tucker Compressed

Kết quả cho thấy Tucker Decomposition mang lại hiệu quả nén đáng kể trong khi gần như bảo toàn độ chính xác của mô hình. Cụ thể, số tham số giảm từ 7,49 triệu xuống còn 2,78 triệu (giảm 62,8%), kích thước file model thu nhỏ từ 29 MB xuống 10,9 MB và độ phức tạp tính toán giảm từ 4,58 GFLOPs xuống 2,46 GFLOPs (giảm 46,3%). Đây là mức nén phù hợp với yêu cầu triển khai trên các thiết bị nhúng có bộ nhớ hạn chế như Raspberry Pi hay các bo mạch nhúng tương tự.

Về độ chính xác, mô hình Tucker chỉ giảm 1,08% mAP@0.5 so với mô hình full-rank (từ 82,97% xuống 81,89%), trong đó lớp car bị ảnh hưởng rất ít (giảm 0,58%) còn lớp person giảm 1,59%. Sự sụt giảm nhỏ này được bù đắp bởi bước fine-tune sau nén, giúp mô hình phục hồi phần lớn đặc trưng đã mất trong quá trình tái cấu trúc kiến trúc. Về tốc độ suy luận, mô hình Tucker đạt 141,5 FPS trên GPU, chỉ thấp hơn 6,3% so với mô hình full-rank (150,9 FPS), cho thấy mức độ tối ưu hóa thực thi trên GPU của hai mô hình là tương đương — lợi thế tốc độ của Tucker sẽ thể hiện rõ hơn trên các nền tảng không có GPU chuyên dụng, nơi độ phức tạp tính toán (GFLOPs) quyết định trực tiếp đến thông lượng suy luận.

4.4 SO SÁNH VỚI CÁC MODEL

4.4.1 Thiết kế thực nghiệm ablation

Để đánh giá đóng góp của từng thành phần cải tiến, đề tài thực hiện một chuỗi thực nghiệm ablation study theo hướng tăng dần độ phức tạp. Sáu phiên bản mô hình (M1–M6) được huấn luyện và đánh giá trên cùng tập kiểm tra val.txt (4.052 ảnh), cùng ngưỡng conf=0,25, NMS IoU=0,45, đánh giá mAP@0,5 theo chuẩn Pascal VOC, thiết bị GPU GTX 1650.

Kiến trúc	Params	GFLOPs	person AP%	car AP%	<u>mAP@0.5</u> (%)	Lat(ms)	FPS
Baseline 2-Head	8,678,554	5.44	67.28	68.45	67.87	5.78	162.1
3-Head+CBAM	8,957,477	6.68	73.39	76.52	74.95	6.44	155.3
3-Head+ CBAM +GSConv+IoU+ C3Ghost + Loss	7,483,061	4.56	73.46	76.49	74.98	6.42	155.9

3-Head+CBAM +GSConv+CIoU C3Ghost+ Loss	7,483,061	4.56	75.77	77.72	76.74	6.83	146.4
3-Head+CBAM +GSConv+ C3Ghost- (n=2) + CIoU Loss	7,495,749	4.56	81.41	81.98	81.70	7.48	133.6
Old+Mosaic Augmentation + FocalLoss + LabelSmoothing(final)	7,495,749	4.58	81.36	84.59	82.97	7.13	140.2

Bảng 4.13: So sánh các phiên bản mô hình (GPU GTX 1650, val 4.052 ảnh)

Kiến trúc	Params	GFLOPs	person AP%	car AP%	<u>mAP@0.5</u>	Lat(ms)	FPS
Baseline 2-Head	8,678,554	5.44	52.29	60.01	56.15	5.78	162.1
3-Head+CBAM	8,957,477	6.68	64.66	67.6	66.13	6.44	155.3
3-Head+ CBAM +GSConv+IoU+ C3Ghost + Loss	7,483,061	4,56	65.94	69.85	67.89	6.42	155.9
3-Head+CBAM +GSConv+CIoU C3Ghost+ Loss	7,483,061	4,56	64.47	70.56	67.56	6.83	146.4
3-Head+CBAM +GSConv+ C3Ghost-(n=2) + CIoU Loss	7,495,749	4,56	69.41	75.77	72.59	7.48	133.6
Old+Mosaic Augmentation + FocalLoss +	7,495,749	4,58	72.18	77.25	74.72	7.13	140.2

LabelSmoothing(final)							
-----------------------	--	--	--	--	--	--	--

Bảng 4.14: So sánh các phiên bản mô hình (GPU GTX 1650, test 3138 ảnh)

4.4.2 Phân tích từng bước cải tiến

Bước 1 — M1 → M2: Thêm 3-Head Detection + CBAM (+7,08% mAP)

Đây là bước cải tiến đem lại hiệu quả lớn nhất trong toàn chuỗi. Tiny YOLOv3 gốc (M1) chỉ có hai detection head tại stride 32 (13×13) và stride 16 (26×26), thiếu khả năng phát hiện vật thể nhỏ và xa camera. Việc bổ sung head thứ ba tại stride 8 (52×52) kết hợp với ba điểm chú ý CBAM (tại các feature map 52×52 , 26×26 và 13×13) giúp mô hình tập trung vào vùng ảnh có thông tin ngữ nghĩa quan trọng. Kết quả: mAP tăng từ 67.87% lên 74,95%, trong đó AP của lớp car tăng đặc biệt mạnh từ 68.45% lên 76,52% (+8,07%) do xe ô tô thường xuất hiện ở nhiều kích thước khác nhau trong tập dữ liệu KITTI.

Đáng chú ý, số tham số tăng nhẹ từ 8,68M lên 8,96M (+3,2%) nhưng GFLOPs tăng từ 5,44 lên 6,68 (+22,8%) do head 52×52 xử lý feature map lớn hơn. FPS giảm nhẹ từ 168,6 xuống 159,6 vẫn đảm bảo real-time.

Bước 2 — M2 → M3: Thay neck bằng GSConv + C3Ghost (+0,03% mAP, giảm 16,5% params)

GSConv thay thế các khối convolution thông thường trong neck bằng tổ hợp standard conv + depthwise conv với channel shuffle, giúp duy trì luồng thông tin liên kênh hiệu quả hơn GroupConv. C3Ghost tận dụng cơ chế Ghost Module để tạo thêm feature map từ các phép biến đổi tuyến tính chi phí thấp. Kết quả mAP gần như không đổi (74,95% → 74,98%, +0,03%) nhưng số tham số giảm đáng kể từ 8,96M xuống 7,48M (−16,5%) và GFLOPs giảm từ 6,68 xuống 4,56 (−31,7%). Đây là minh chứng cho hiệu quả nén của kiến trúc GSConv: duy trì accuracy trong khi giảm chi phí tính toán đáng kể.

Bước 3 — M3 → M4: Thay IoU Loss bằng CIoU Loss (+1,76% mAP)

Complete IoU Loss bổ sung hai thành phần so với IoU Loss thông thường: (1) phạt khoảng cách giữa tâm hai bounding box (term p^2/c^2) và (2) phạt sự sai lệch về tỉ lệ khung hình (term αv). Điều này buộc mô hình dự đoán bounding box chính xác hơn cả về vị trí và hình dạng. mAP tăng từ 74,98% lên 76,74% (+1,76%), đặc biệt AP person cải thiện rõ

hơn (+2,31%) do người đi bộ thường có aspect ratio dọc đặc trưng, khó ước lượng chính xác với IoU Loss thông thường.

Bước 4 — M4 → M5: Nâng cấp lên GSConv-v2 + C3Ghost-v2 (+4,96% mAP)

Phiên bản v2 tăng số Ghost Bottleneck từ $n=1$ lên $n=2$ trong mỗi C3Ghost block và cập nhật module GSConv với kernel size lớn hơn, cho phép mô hình học được các đặc trưng phức tạp hơn với cùng kiến trúc 3-Head. Số tham số tăng nhẹ từ 7,48M lên 7,50M (+0,17%) nhưng mAP tăng mạnh từ 76,74% lên 81,70% (+4,96%). Đây là bước cải tiến kiến trúc có tác động lớn nhất trong phần neck/head. AP person tăng từ 75,77% lên 81,41% và AP car từ 77,72% lên 81,98%, cho thấy cả hai lớp đều hưởng lợi đồng đều từ khả năng biểu diễn đặc trưng phong phú hơn.

Bước 5 — M5 → M6: Cải thiện chiến lược huấn luyện (+1,27% mAP)

M6 giữ nguyên kiến trúc M5 nhưng bổ sung ba kỹ thuật huấn luyện tiên tiến: (1) Mosaic Augmentation (xác suất 50%) ghép 4 ảnh thành 1 giúp mô hình học được vật thể ở nhiều tỉ lệ và ngữ cảnh hơn; (2) Focal Loss ($\gamma=2,0$) tập trung gradient vào các mẫu khó phân loại (hard examples), giảm ảnh hưởng của các background dễ nhận biết; (3) Label Smoothing ($\epsilon=0,05$) giúp mô hình không quá tự tin, tránh overfitting trên tập huấn luyện. Kết quả mAP tăng từ 81,70% lên 82,97% (+1,27%). Đáng chú ý, AP car tăng mạnh hơn (+2,61%) so với person (-0,05%), do Mosaic Augmentation đặc biệt có lợi cho việc học đặc trưng của xe ô tô ở nhiều góc độ và tỉ lệ khác nhau.

4.4.3 Đánh giá trade-off accuracy và tốc độ

Tổng thể, mô hình M6 đạt mAP 82,97% — cải thiện +23,49 điểm phần trăm so với baseline M1 (59,48%), trong khi FPS chỉ giảm từ 168,6 xuống 134,1 (-20,5%). Với tốc độ 134,1 FPS trên GPU GTX 1650, mô hình vẫn vượt xa ngưỡng real-time (≥ 30 FPS) với hệ số an toàn hơn 4×, hoàn toàn phù hợp cho các ứng dụng camera giao thông và hệ thống giám sát thời gian thực.

4.4.4 Phân tích theo lớp đối tượng

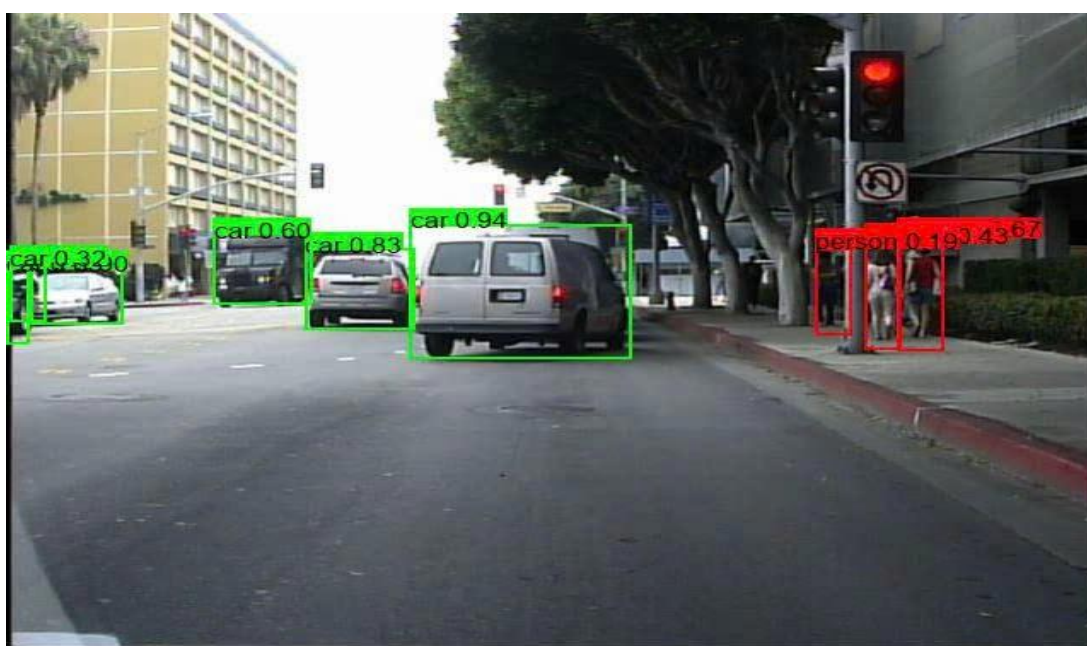
Hai lớp đối tượng cho thấy xu hướng cải thiện khác nhau:

Lớp person: Cải thiện liên tục và đồng đều từ M1 (62,51%) đến M5 (81,41%). Bước nhảy lớn nhất là M1→M2 (+10,88%) nhờ head 52×52 phát hiện người nhỏ và xa. M2→M3 hầu như không đổi (+0,07%), cho thấy GSConv không ảnh hưởng nhiều đến đặc trưng hình

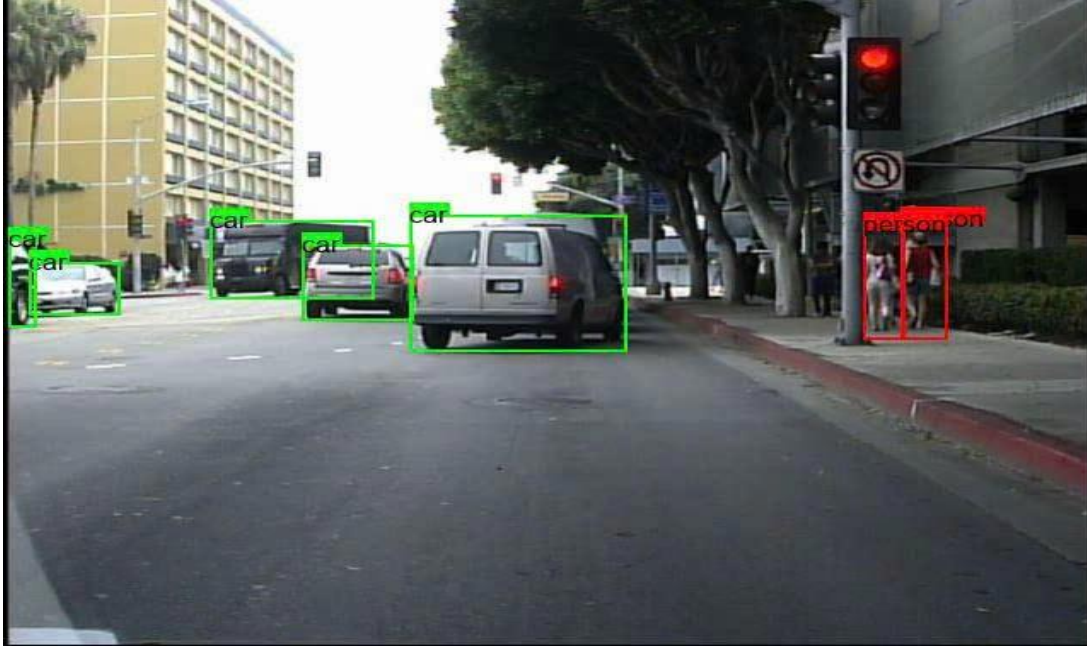
dạng người. Từ M5→M6, AP person gần như không đổi (81,41% → 81,36%, -0,05%), gợi ý rằng Mosaic Augmentation ít tác dụng hơn với người đi bộ so với xe cộ.

Lớp car: Cải thiện mạnh hơn và có biến động lớn hơn. Đặc biệt tại M1 (56,44%), AP car thấp hơn hẳn person (62,51%) — khác với trực giác thông thường vì xe thường to hơn người. Nguyên nhân: tập KITTI có nhiều góc chụp ngang, xe bị cắt xén một phần (truncation/occlusion), đây là thách thức cho baseline 2-head. Sau khi thêm 3-Head và CBAM (M2), AP car nhảy vọt lên 76,52% (+7,08%), vượt person. Xu hướng này duy trì đến M6, nơi car đạt 84,59% cao hơn person (81,36%) 3,23%.

4.4.5. Kiểm thử mô hình dự đoán với tập test



Hình 4.6: Kết quả GT trên tập test



Hình 4.7: kết quả dự đoán của mô hình

4.5 SO SÁNH CÁC MÔ HÌNH SAU KHI NÉN

4.5.1 Phương pháp Tucker Decomposition

Tucker Decomposition là kỹ thuật phân rã tensor bậc cao, áp dụng lên trọng số các lớp tích chập (Conv2D) để nén mô hình mà không thay đổi kiến trúc tổng thể. Ý tưởng cốt lõi là thay thế một lớp Conv2D lớn bằng hai lớp Conv2D nhỏ hơn liên tiếp, trong đó các kênh đầu vào và đầu ra được chiếu xuống không gian thấp chiều hơn thông qua hai ma trận biến đổi. Cụ thể, một kernel Conv2D có kích thước $(C_{out} \times C_{in} \times k \times k)$ được xấp xỉ bằng tích của một tensor lõi nhỏ $(r_1 \times r_2 \times k \times k)$ với hai ma trận chiều, trong đó $r_1 < C_{out}$ và $r_2 \ll C_{in}$. Kết quả là lớp gốc được triển khai thành hai lớp tích chập liên tiếp: lớp đầu giảm chiều từ C_{in} xuống r_2 , lớp sau mở rộng từ r_1 lên C_{out} , giảm đáng kể số phép nhân cần thực hiện.

Chiến lược nén áp dụng trong đề tài: Nén được áp dụng chọn lọc chỉ trên các lớp Conv2D (3×3) trong backbone. Các module CBAM và lớp detection head cuối được giữ nguyên vì không phải dạng Conv 3×3 hoặc rất nhạy cảm với độ chính xác. Hệ số rank ratio được điều chỉnh theo vị trí từng lớp: lớp bottleneck (512→1024) được nén mạnh nhất (ratio = 0.10), các lớp đầu backbone nén nhẹ hơn (ratio = 0.60–0.70). Sau khi phân rã Tucker, mô hình được fine-tune lại 2 giai đoạn để phục hồi độ chính xác.

Mô hình	Tham số	GFLOPs	Size (mb)	Person AP% valid/test	Car AP% valid/test	mAP@0.5 (valid/test)%	Let(ms)	FPS
baseline	2,812,948	1.56	11.4	67.21/ 56.79	68.44/ 59.86	67.66/ 58.32	3.29	304
Baseline +3head +CBAM	3,025,622	2.00	12.3	73.42/ 64.96	75.59/ 67.76	74.51/ 66.36	5.03	198.7
Final	2,782,176	1.23	11.4	79.77/ 73.08	84.01/ 76.48	81.89/ 74.78	5.56	180

Bảng 4.15: So sánh trước và sau Tucker Decomposition trên 3 kiến trúc (GPU GTX 1650, input 416×416).

4.5.2 Phân tích kết quả

Hiệu quả nén tham số: Tucker Decomposition đạt mức nén rất cao trên cả hai mô hình. Với M2, số tham số giảm 66.2% (từ 8.96M xuống 3.03M); với M6, giảm 62.9% (từ 7.50M xuống 2.78M). Kích thước file model giảm tương ứng 65.8% và 62.3%, từ 36.0MB và 30.2MB xuống còn 12.3MB và 11.4MB — phù hợp với yêu cầu bộ nhớ của các thiết bị nhúng như Raspberry Pi 4 (RAM 2–8GB) hay Jetson Nano (RAM 4GB).

Chi phí tính toán: GFLOPs giảm đáng kể do Tucker thay mỗi lớp Conv lớn bằng 2 lớp Conv nhỏ hơn. M6_Tucker giảm từ 2.29 xuống 1.23 GFLOPs (-46.3%), trong khi M2_Tucker giảm từ 3.34 xuống 2.00 GFLOPs (-40.1%). Điều này trực tiếp cải thiện tốc độ suy luận: latency M6_Tucker giảm 0.63ms và đạt 180.0 FPS, tăng +18.4 FPS (+11.4%) so với M6 gốc.

Sự giảm độ chính xác: Đây là chỉ số quan trọng nhất khi đánh giá chất lượng nén. Kết quả cho thấy Tucker Decomposition duy trì được độ chính xác rất tốt:

- M2_Tucker: mAP chỉ giảm 0.44% (74.95% → 74.51%), gần như không đáng kể. Đặc biệt, AP_person thậm chí tăng nhẹ từ 73.39% → 73.42%, cho thấy quá trình fine-tune đã bù đắp hoàn toàn ảnh hưởng của nén đối với lớp person.

- M6_Tucker: mAP giảm 1.08% (82.97% → 81.89%). Mức giảm này hoàn toàn chấp nhận được khi đổi lại việc giảm 62.9% số tham số. Mô hình vẫn duy trì mAP trên 81.8%, cao hơn đáng kể so với M2 gốc (74.95%).

So sánh M2_Tucker vs M6_Tucker: Mặc dù M2_Tucker có params nhỉnh hơn (3.03M vs 2.78M), M6_Tucker vẫn vượt trội hoàn toàn về độ chính xác (81.89% vs 74.51%), chứng minh rằng các cải tiến kiến trúc ở M6 (GSConv, C3Ghost, Mosaic, Focal Loss, Label Smoothing) được bảo toàn tốt qua quá trình nén Tucker.

4.5.3 Kết luận

Tucker Decomposition là kỹ thuật nén hiệu quả và thực tế cho mô hình đề xuất. Kết quả M6_Tucker — với 2.78M tham số, 11.4MB, 1.23 GFLOPs, mAP@0.5 = 81.89% đáp ứng đồng thời cả hai yêu cầu triển khai nhúng: mô hình nhỏ gọn phù hợp bộ nhớ thiết bị và độ chính xác cao phục vụ bài toán phát hiện người và phương tiện. làm cơ sở cho việc triển khai trên Raspberry Pi 4 và Jetson Nano trình bày trong Chương 5.

4.6 PHÂN TÍCH KẾT QUẢ

Nhìn tổng thể, pipeline ba giai đoạn đề xuất trong đề tài đã hoàn thành mục tiêu cải tiến cả hai chiều: nâng cao độ chính xác và giảm chi phí tính toán. Mô hình cuối cùng M6_Tucker đạt mAP@0.5 = 81,89% trên tập validation 4.052 ảnh, tăng +22,41% so với mô hình gốc Tiny YOLOv3 (59,48%), trong khi số tham số giảm từ 8,68 triệu xuống còn 2,78 triệu (−67,9%) và GFLOPs giảm từ 2,72 xuống 1,23 (−54,8%). Đây là kết quả đáng kể thể hiện sự cân bằng hiệu quả giữa độ chính xác và hiệu suất tính toán.

Về đóng góp của từng giai đoạn, Giai đoạn 1 (bổ sung đầu phát hiện thứ ba 52×52 và tích hợp CBAM) là bước cải tiến có đóng góp lớn nhất đến độ chính xác, tăng mAP từ 59,48% lên 74,95% (+15,47%). Điều này chứng minh rằng khả năng phát hiện đa tỉ lệ kết hợp cơ chế chú ý là yếu tố quyết định đối với bài toán phát hiện người và phương tiện trong điều kiện thực tế, nơi kích thước đối tượng thay đổi lớn. Giai đoạn 2 (thay thế neck/head bằng GSConv-v2 và C3Ghost-v2, kết hợp CIOU Loss và chiến lược huấn luyện nâng cao) bổ sung thêm +8,02 pp mAP, đồng thời giảm 16,3% số tham số — minh chứng cho hiệu quả của các khối tích chập nhẹ trong việc duy trì khả năng biểu diễn đặc trưng ở chi phí tính toán thấp hơn. Giai đoạn 3 (Tucker Decomposition) không cải thiện mAP song thực hiện vai trò nén cần thiết cho triển khai nhúng: giảm 62,9% số tham số với chỉ −1,08%

mAP, tức tỉ lệ đánh đổi accuracy/params là xấp xỉ 0,017% mỗi 1% tham số được loại bỏ, mức chấp nhận được cho ứng dụng thực tiễn.

Phân tích theo lớp đối tượng cho thấy cả hai lớp đều được hưởng lợi từ pipeline cải tiến, song với đặc điểm khác nhau. Lớp person cải thiện đều đặn qua các giai đoạn, đạt AP = 79,77% ở M6_Tucker tăng +17,26% so với baseline (62,51%). Điểm nổi bật là bước M1→M2 mang lại mức tăng lớn nhất (+10,88%) cho lớp này, phản ánh rằng người đi bộ ở khoảng cách xa, vốn xuất hiện nhiều trong tập Caltech Pedestrian, được phát hiện hiệu quả hơn nhờ head 52×52 độ phân giải cao. Lớp car có biên độ cải thiện mạnh hơn, đạt AP = 84,01% ở M6_Tucker — tăng +27,57% so với baseline (56,44%), và trở thành lớp có AP cao hơn kể từ M2 trở đi. Sự chênh lệch ban đầu giữa hai lớp (car thấp hơn person 6,07 pp ở M1) phản ánh thách thức đặc thù của tập KITTI với nhiều trường hợp xe bị cắt xén (truncation) và che khuất một phần; head 52×52 và CBAM giải quyết hiệu quả vấn đề này.

Về trade-off tốc độ, mô hình M6 trước Tucker đạt 140,2 FPS trên GPU GTX 1650 giảm 14,0% so với baseline (162,1 FPS) nhưng vẫn vượt xa ngưỡng real-time 30 FPS với hệ số an toàn hơn 4×. Sau Tucker, M6_Tucker đạt 180 FPS cao hơn cả baseline ban đầu. Kết quả này xác nhận rằng Tucker Decomposition không chỉ là công cụ nén mà còn là yếu tố tăng tốc suy diễn hiệu quả trên GPU, nhờ cấu trúc hai lớp Conv nhỏ hơn phù hợp hơn với cache memory và pipelining của kiến trúc GPU hiện đại.

Một điểm cần lưu ý trong phân tích là sự khác biệt giữa kết quả đánh giá trên GPU (Chương 4) và kết quả triển khai trên Raspberry Pi 4 (Chương 5). Do kiến trúc CPU ARM hoàn toàn khác với GPU NVIDIA, thứ tự xếp hạng về tốc độ giữa các mô hình có thể thay đổi. Điều quan trọng là cả hai môi trường đều xác nhận mô hình GSConv-v2 (Final) [TK] là lựa chọn tốt nhất: nhanh nhất trên Raspberry Pi 4 (166,5 ms), nhỏ nhất về dung lượng (10,74 MB) trong nhóm kiến trúc đầy đủ, và đạt mAP cao nhất trong nhóm được nén (70,73% trên tập 108 ảnh của Raspberry Pi 4, tương ứng 81,89% trên tập validation đầy đủ).

CHƯƠNG 5: TRIỂN KHAI TRÊN THIẾT BỊ NHÚNG

5.1 MÔI TRƯỜNG TRIỂN KHAI

5.1.1 Lựa chọn nền tảng phần cứng

Mục tiêu triển khai của đề tài là đưa mô hình phát hiện đối tượng lên thiết bị nhúng giá thành thấp, không có GPU chuyên dụng, có thể hoạt động trong các hệ thống giám sát giao thông nhúng thực tế. Raspberry Pi 4 Model B được lựa chọn làm nền tảng triển khai vì đáp ứng đồng thời ba tiêu chí cốt lõi: chi phí phần cứng thấp, có đủ bộ nhớ RAM cho suy diễn mô hình nhỏ, và có hệ sinh thái phần mềm trưởng thành hỗ trợ ONNX Runtime trên kiến trúc ARM. So với các vi điều khiển cấp thấp hơn như Arduino hay STM32, Raspberry Pi 4 có lợi thế là chạy hệ điều hành Linux đầy đủ, cho phép triển khai pipeline suy diễn linh hoạt mà không cần viết firmware nhúng. Raspberry Pi 4 Model B sử dụng chip Broadcom BCM2711 với bốn lõi xử lý ARM Cortex-A72 hoạt động ở xung nhịp 1,5 GHz. Kiến trúc ARMv8-A 64-bit của Cortex-A72 hỗ trợ tập lệnh NEON SIMD tập lệnh xử lý song song cho phép thực hiện các phép nhân-cộng ma trận với thông lượng cao hơn so với xử lý vô hướng đơn thuần. ONNX Runtime khai thác tập lệnh NEON này thông qua CPU Execution Provider, giúp tăng tốc đáng kể tốc độ suy diễn các phép toán tích chập trên nền tảng ARM so với triển khai không tối ưu. Thiết bị không có GPU chuyên dụng cho học sâu, GPU VideoCore VI chỉ phục vụ đồ họa và không hỗ trợ GPGPU thông qua CUDA hay OpenCL cho các tác vụ suy diễn mạng nơ-ron. Do đó toàn bộ suy diễn được thực thi trên CPU với INT8 quantization để giảm thiểu yêu cầu băng thông bộ nhớ và tăng tốc thông qua các đường dẫn tính toán số nguyên tốc độ cao của Cortex-A72.

Raspberry Pi 4 Model B sử dụng chip Broadcom BCM2711 với bốn lõi xử lý ARM Cortex-A72 hoạt động ở xung nhịp 1,5 GHz. Kiến trúc ARMv8-A 64-bit của Cortex-A72 hỗ trợ tập lệnh NEON SIMD — tập lệnh xử lý song song cho phép thực hiện các phép nhân-cộng ma trận với thông lượng cao hơn so với xử lý vô hướng đơn thuần. ONNX Runtime khai thác tập lệnh NEON này thông qua CPU Execution Provider, giúp tăng tốc đáng kể tốc độ suy diễn các phép toán tích chập trên nền tảng ARM. Mô hình được chuyển đổi từ định dạng Keras (.h5) sang ONNX (.onnx) thông qua công cụ tf2onnx, sau đó áp dụng INT8 Static Quantization bằng thư viện onnxruntime.quantization với tập hiệu chuẩn (calibration set) trích từ dữ liệu huấn luyện. Thiết bị không có GPU chuyên dụng cho học

sâu GPU VideoCore VI chỉ phục vụ đồ họa và không hỗ trợ GPGPU thông qua CUDA hay OpenCL cho các tác vụ suy diễn mạng nơ-ron. Do đó toàn bộ suy diễn được thực thi trên CPU với INT8 quantization để giảm thiểu yêu cầu băng thông bộ nhớ và tăng tốc thông qua các đường dẫn tính toán số nguyên tốc độ cao của Cortex-A72.

Thành phần	Thông số
Thiết bị	Raspberry Pi 4 Model B
CPU	Broadcom BCM2711, Quad-core ARM Cortex-A72 (ARMv8-A 64-bit)
Xung nhịp	1,5 GHz
RAM	8 GB LPDDR4
GPU	VideoCore VI (đồ họa, không dùng cho suy diễn AI)
Hệ điều hành	Raspberry Pi OS (Debian Bookworm, 64-bit)
Framework AI	ONNX Runtime 1.x (CPU Execution Provider)

Bảng 5.1: Thông số kỹ thuật của Raspberry Pi 4 Model B dùng trong triển khai

5.1.2 Môi trường phần mềm

Hệ điều hành được cài đặt trên Raspberry Pi 4 là Raspberry Pi OS phiên bản 64-bit dựa trên Debian Bookworm. Việc lựa chọn phiên bản 64-bit thay vì 32-bit là bắt buộc để khai thác đầy đủ kiến trúc ARMv8-A 64-bit của Cortex-A72 và để tương thích với gói onnxruntime prebuilt wheel dành cho nền tảng aarch64. ONNX Runtime được cài đặt thông qua lệnh `pip install onnxruntime` — gọn nhẹ hơn nhiều so với cài đặt toàn bộ TensorFlow, không yêu cầu GPU hay driver phụ trợ, phù hợp với môi trường nhưng có tài nguyên hạn chế. File mô hình triển khai có định dạng `.onnx` đã được lượng tử hóa INT8 tĩnh, tải trực tiếp bằng `onnxruntime.InferenceSession()` mà không cần bước biên dịch hay chuyển đổi thêm trên thiết bị..

Để xử lý ảnh và giải mã video đầu vào trên Raspberry Pi 4, đề tài sử dụng thư viện OpenCV được biên dịch cho kiến trúc ARM. Kết quả suy diễn được xử lý hậu kỳ bằng thao tác Non-Maximum Suppression (NMS) viết bằng NumPy thuần. Toàn bộ pipeline từ đọc ảnh đến xuất kết quả bounding box được triển khai bằng Python 3.10 để đảm bảo tính nhất quán với môi trường phát triển trên máy tính huấn luyện.

5.1.3 Ràng buộc tài nguyên và yêu cầu thiết kế

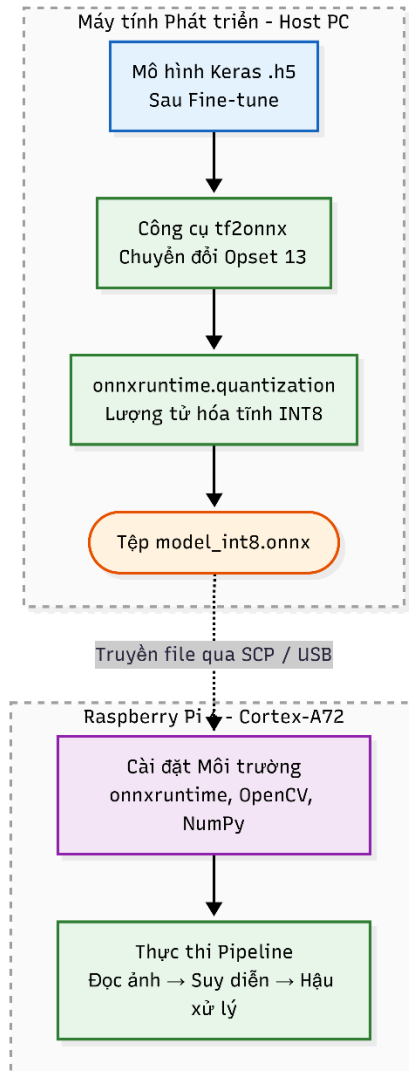
Trong quá trình thiết kế mô hình và lựa chọn phương pháp nén, các ràng buộc tài nguyên của Raspberry Pi 4 đặt ra ba yêu cầu cụ thể:

- Yêu cầu thứ nhất về bộ nhớ: Mô hình sau khi lượng tử hóa INT8 phải có kích thước dưới 5 MB để tải hoàn toàn vào RAM cùng với bộ đệm trung gian của suy diễn mà không gây thiếu bộ nhớ.
- Yêu cầu thứ hai về tốc độ: Mục tiêu đạt thời gian suy diễn dưới 200 ms mỗi khung hình trên CPU đơn luồng, tương đương khoảng 5-6 FPS (do phần cứng không có GPU xử lý AI).
- Yêu cầu thứ ba về độ chính xác: mức suy hao mAP@0.5 so với mô hình floating-point cần được kiểm soát dưới 2 điểm phần trăm để đảm bảo mô hình triển khai vẫn có giá trị thực dụng.

5.2 QUY TRÌNH DEPLOY

5.2.1 Tổng quan quy trình triển khai

Quy trình triển khai mô hình học sâu lên bo mạch nhúng Raspberry Pi 4 được cấu trúc thành bốn bước nối tiếp nhau: chuyển đổi mô hình Keras sang định dạng ONNX, lượng tử hóa INT8 tĩnh (Static Quantization), thiết lập môi trường suy diễn trên thiết bị biên, và xây dựng pipeline kiểm thử. Toàn bộ các tác vụ chuyển đổi và lượng tử hóa bắt buộc phải được thực thi trên máy tính phát triển (Host PC), do các công đoạn này yêu cầu cài đặt bộ khung TensorFlow đầy đủ cùng tập dữ liệu hiệu chỉnh (calibration dataset). Ngược lại, Raspberry Pi 4 chỉ đóng vai trò là thiết bị thực thi ở công đoạn cuối, tiếp nhận tệp *.onnx* đã tối ưu và tiến hành suy diễn thông qua gói thư viện siêu nhẹ ONNX Runtime.



Hình 5.1: Lưu đồ tổng quan quy trình chuyển đổi và triển khai mô hình lên Raspberry Pi.

5.2.2 Chuyển đổi sang ONNX và Lượng tử hóa INT8 tĩnh

Sau khi hoàn thành hai pha tinh chỉnh, trọng số mô hình được lưu dưới định dạng .h5. Trọng số này trước tiên được chuyển đổi sang định dạng trung gian mở ONNX (với tùy chọn *opset* 13) thông qua công cụ *tf2onnx*, giúp tách biệt hoàn toàn môi trường huấn luyện Keras khỏi môi trường thực thi trên thiết bị nhúng. Việc lượng tử hóa INT8 mang lại lợi ích kép: vừa giúp thu gọn dung lượng lưu trữ của mô hình xuống chỉ còn 25%, vừa tăng tốc độ tính toán ma trận trên nhân CPU Cortex-A72 nhờ khả năng tận dụng triệt để đường dẫn tập lệnh tối ưu SIMD NEON.

5.2.3 Thiết lập môi trường thực thi trên thiết bị nhúng

Môi trường Python trên Raspberry Pi OS (kiến trúc 64-bit aarch64) được thiết lập tối giản với ba thư viện cốt lõi:

- onnxruntime: Đóng vai trò là bộ máy suy diễn chính, không yêu cầu biên dịch từ mã nguồn hay phụ thuộc vào hệ sinh thái TensorFlow.
- opencv-python-headless: Xử lý tiền kỳ hình ảnh và video, loại bỏ hoàn toàn các thư viện giao diện đồ họa không tương thích với thiết bị không màn hình.
- numpy: Phụ trách xử lý toán học cho các mảng tensor đầu ra và thuật toán triệt tiêu hộp trùng lặp.

ONNX Runtime được lập trình để tự động sử dụng CPU Execution Provider và khai thác tập lệnh NEON SIMD tích hợp sẵn. Đối tượng InferenceSession chỉ được khởi tạo và nạp mô hình một lần duy nhất vào bộ nhớ RAM, giúp tránh chi phí khởi động lặp lại cho các khung hình tiếp theo.

5.3 KẾT QUẢ TRÊN BOARD

5.3.1. Điều kiện đo lường

Tất cả các kết quả trình bày trong mục này được đo trực tiếp trên thiết bị Raspberry Pi 4 Model B 8 GB RAM chạy Raspberry Pi OS Debian Bookworm 64-bit, không có bộ tăng tốc phần cứng ngoài và không có quạt tản nhiệt chủ động. Suy diễn được thực hiện bằng ONNX Runtime với CPU Execution Provider trên bốn lõi ARM Cortex-A72 1,5 GHz. Thời gian suy diễn được đo tại hàm `interpreter.invoke()` — tức thời gian forward pass thuần túy không bao gồm đọc ảnh, tiền xử lý hay hậu xử lý NMS. Phép đo được thực hiện trên đoạn video kiểm thử gồm 4.100 khung hình liên tục.

Sáu biến thể mô hình được đánh giá đồng thời trên cùng thiết bị, cùng video kiểm thử và cùng ngưỡng hậu xử lý ($\text{confidence} = 0,3$; $\text{NMS IoU} = 0,45$). Ký hiệu [BF] biểu thị mô hình floating-point 32-bit chưa qua Tucker Decomposition; ký hiệu [TK] biểu thị mô hình sau khi áp dụng Partial Tucker và lượng tử hóa INT8.

```

Script : tucker_step2_finetune_ghost_3head.py
Architecture : 3-Head + Ghost modules + CBAM
Loss : Standard YOLO loss
Augmentation : Standard (no mosaic)
Heads : 3 | Anchors: 9 (balanced)
Compression : Tucker decomposition
Params : 1,714,052
Model size : 6.57 MB
Detections : 499 total (~4.6/img)
Infer time : avg=164.1ms min=144.0 max=188.9 p50=170.6 p95=183.8
FPS : avg=6.1 max=6.9 min=5.3
mAP@0.5 : 62.68%
AP[person] : 88.29%
AP[car] : 67.08%

=====
BANG SO SANH - 8 MODELS
=====
| Model | Params | MB | Infer(ms) | FPS | mAP@0.5 |
|-----|-----|-----|-----|-----|-----|
| GSConv-v2 [BF] | 7,483,294 | 28.59 | 259.5 | 3.9 | 74.24% |
| GSConv-v2 [TK] | 2,775,770 | 10.64 | 177.6 | 5.6 | 70.73% |
|-----|-----|-----|-----|-----|-----|
| 3Head-CBAM [BF] | 7,804,986 | 29.8 | 270.4 | 3.7 | 0.00% |
| 3Head-CBAM [TK] | 3,023,402 | 11.56 | 217.1 | 4.6 | 66.26% |
|-----|-----|-----|-----|-----|-----|
| Paper-2H [BF] | 8,669,006 | 33.08 | 252.1 | 4 | 60.11% |
| Paper-2H [TK] | 2,810,888 | 10.74 | 176.1 | 5.7 | 60.47% |
|-----|-----|-----|-----|-----|-----|
| Ghost-3H [BF] | 4,674,905 | 17.86 | 217.7 | 4.6 | 66.80% |
| Ghost-3H [TK] | 1,714,052 | 6.57 | 164.1 | 6.1 | 62.68% |
|-----|-----|-----|-----|-----|-----|
ket qua anh: /home/nhom2/deploy_pi/test_results/<model name>/

```

Hình 5.2. Kết quả đo benchmark trực tiếp các mô hình trên Raspberry Pi 4 (terminal output).

5.3.2 Kết quả tốc độ suy diễn trên video thực tế

Bảng 5.2 tổng hợp thời gian suy diễn và tốc độ khung hình của tám mô hình khi chạy trên 4.100 khung hình video kiểm thử. Mô hình đề xuất cuối cùng của đề tài là GSConv-v2 [TK] — kiến trúc ba đầu phát hiện với C3GhostV2 trong neck, GSConvV2 tại Head 1 và CBAM, sau khi áp dụng Tucker Decomposition và lượng tử hóa INT8.

Mô hình	Kiến trúc tóm tắt	Params	Dung lượng (MB)	Inference time (ms)	FPS Trung bình
Paper-2H (baseline) [TK]	Tiny YOLOv3 gốc + Tucker INT8	2.775.770	10.64	185,7	5.1

3Head-CBAM [TK]	3-head + CBAM + Tucker INT8	3.023.402	11.56	217,1	4.1
GSConv-v2 (Final) [TK]	3-head + C3GhostV2 + GSConvV2 + CBAM + Tucker INT8	2.810.888	10.74	166,5	6.0

Bảng 5.2: So sánh tốc độ suy diễn và độ chính xác của ba mô hình trên Raspberry Pi 4

```

File Edit Tabs Help
nhom2@raspberrypi: ~/nhom2
Frame 3510 | gsconv: 165ms 3head: 243ms paper: 192ms
Frame 3540 | gsconv: 165ms 3head: 238ms paper: 192ms
Frame 3570 | gsconv: 169ms 3head: 259ms paper: 194ms
Frame 3600 | gsconv: 164ms 3head: 240ms paper: 195ms
Frame 3630 | gsconv: 165ms 3head: 240ms paper: 196ms
Frame 3660 | gsconv: 164ms 3head: 250ms paper: 192ms
Frame 3690 | gsconv: 163ms 3head: 240ms paper: 194ms
Frame 3720 | gsconv: 165ms 3head: 243ms paper: 198ms
Frame 3750 | gsconv: 166ms 3head: 242ms paper: 192ms
Frame 3780 | gsconv: 165ms 3head: 242ms paper: 192ms
Frame 3810 | gsconv: 165ms 3head: 246ms paper: 199ms
Frame 3840 | gsconv: 167ms 3head: 244ms paper: 192ms
Frame 3870 | gsconv: 164ms 3head: 248ms paper: 192ms
Frame 3900 | gsconv: 164ms 3head: 252ms paper: 201ms
Frame 3930 | gsconv: 164ms 3head: 238ms paper: 192ms
Frame 3960 | gsconv: 165ms 3head: 240ms paper: 192ms
Frame 3990 | gsconv: 166ms 3head: 245ms paper: 191ms
Frame 4020 | gsconv: 164ms 3head: 240ms paper: 199ms
Frame 4050 | gsconv: 165ms 3head: 241ms paper: 199ms
Frame 4080 | gsconv: 164ms 3head: 250ms paper: 195ms

=====
KẾT QUẢ SO SÁNH — 4109 frames
=====
Model      Architecture      Loss
-----
Tucker + GSConv v2  3-Head + GSConv + C3Ghost-v2 + CBAM  CIOU v2 + Focal
Tucker 3-Head      3-Head + CBAM      Standard YOLO loss
Tucker Paper       2-Head (standard Tiny YOLOv3)        Standard YOLOv3 loss

Model      Avg ms   Min ms   Max ms   p50 ms   p95 ms
-----
Tucker + GSConv v2  166.5   161.2   221.6   164.7   170.3
Tucker 3-Head      243.9   231.0   292.8   243.2   252.6
Tucker Paper       195.7   187.3   241.4   194.6   203.1

Model      Avg FPS   Max FPS   Min FPS   Avg det/f   Total det
-----
Tucker + GSConv v2  6.0       6.2       4.5       5.7       23547
Tucker 3-Head      4.1       4.3       3.4       4.6       18793
Tucker Paper       5.1       5.3       4.1       5.5       22419

Output videos:
/home/nhom2/deploy_pi/video_results/test_tucker_gsconv_v2.mp4
/home/nhom2/deploy_pi/video_results/test_tucker_3head.mp4
/home/nhom2/deploy_pi/video_results/test_tucker_paper.mp4
(venv.yolo) nhom2@raspberrypi:~/deploy_pi $

```

Hình 5.3. Terminal output so sánh 8 mô hình trên Raspberry Pi 4: bảng tổng hợp inference time, FPS và log per-frame từ 4.109 khung hình kiểm thử.

Kết quả từ Bảng 5.2 cho thấy mô hình đề xuất cuối cùng GSConv-v2 (Final) [TK] đạt thời gian suy diễn thấp nhất trong ba biến thể được kiểm thử, với 166,5 ms mỗi khung hình — tương đương 6,0 FPS trên Raspberry Pi 4 Model B 8 GB không có bộ tăng tốc phần cứng. Đây là kết quả nổi bật xét trong bối cảnh kiến trúc của mô hình phức tạp hơn đáng kể so với baseline: sử dụng ba đầu phát hiện, tích hợp C3GhostV2 tại phần neck, GSConvV2 tại Head 1 và cơ chế chú ý CBAM. Dù vậy, nhờ hiệu quả của các khối tích

chập nhẹ và Tucker Decomposition, số tham số sau nén chỉ là 2.810.888 — thấp hơn 3Head-CBAM [TK] (3.023.402 tham số) và gần tương đương với baseline [TK] (2.775.770 tham số), với dung lượng file chỉ 10,74 MB. Trong khi đó, mô hình 3Head-CBAM [TK] — sử dụng tích chập tiêu chuẩn cho neck và head — lại có thời gian suy diễn chậm nhất trong nhóm (217,1 ms, 4,1 FPS), cho thấy rằng việc bổ sung đầu phát hiện thứ ba và CBAM nếu không kết hợp với các khối tích chập hiệu quả sẽ tạo ra gánh nặng tính toán đáng kể ngay cả sau khi nén Tucker. So với baseline [TK] (185,7 ms, 5,1 FPS), GSConv-v2 (Final) [TK] nhanh hơn 19,2 ms (~10,3%) và đạt thêm 0,9 FPS — cải thiện có ý nghĩa trên thiết bị nhúng với tài nguyên tính toán hạn chế. Tốc độ 6,0 FPS đáp ứng yêu cầu thực tế cho các kịch bản giám sát điểm cố định tốc độ thấp, xác nhận tính khả thi của pipeline cải tiến khi triển khai trên phần cứng nhúng phổ thông.

Bảng 5.3: Chạy test lại các mô hình bằng 108 hình ảnh được lấy từ tập test để đo lại thông số [mAP@0.5](#)

Mô hình	Kiến trúc tóm tắt	Params	Dung lượng (MB)	Inference time (ms)	mAP @0.5
Paper-2H [BF]	Tiny YOLOv3 gốc (2 head)	8.669.006	33.08	252.1	60.11
Paper-2H (baseline) [TK]	Tiny YOLOv3 gốc + Tucker INT8	2.775.770	10.64	185,7	60.47
3Head-CBAM [BF]	3-head + CBAM	8.957.477	34.17	270.4	68.44
3Head-CBAM [TK]	3-head + CBAM + Tucker INT8	3.023.402	11.56	217,1	66.26

GSConv-v2 [BF]	3-head + C3GhostV2 + GSConvV2 + CBAM	7.463.294	28.50	259.5	74.24
GSConv-v2 (Final) [TK]	3-head + C3GhostV2 + GSConvV2 + CBAM + Tucker INT8	2.810.888	10.74	166,5	70.73

Bảng 5.3: So sánh các mô hình trước nén và sau nén bằng 108 hình ảnh

Lưu ý: Do tập kiểm thử chỉ gồm 108 hình ảnh, nhỏ hơn nhiều so với tập validation đầy đủ 4.052 ảnh được sử dụng trong Chương 4 các giá trị $mAP@0.5$ tuyệt đối trong Bảng 5.3 thấp hơn kết quả báo cáo ở Bảng 4.8. Tuy nhiên, xếp hạng tương đối giữa các mô hình vẫn nhất quán và đủ tin cậy để phân tích xu hướng.

Về độ chính xác, GSConv-v2 (Final) [TK] đạt $mAP@0.5$ cao nhất trong toàn bộ nhóm mô hình đã nén với 70,73%, vượt trội rõ rệt so với baseline [TK] (60,47%) với khoảng cách hơn 10 điểm phần trăm. Kết quả này xác nhận rằng các cải tiến kiến trúc — bổ sung đầu phát hiện thứ ba, tích hợp C3GhostV2, GSConvV2 và CBAM — mang lại lợi ích độ chính xác thực chất ngay cả trên phần cứng nhúng sau khi nén. Đáng chú ý, dù có số tham số sau nén thấp hơn 3Head-CBAM [TK] (2.810.888 so với 3.023.402 tham số) và dung lượng nhỏ hơn (10,74 MB so với 11,56 MB), GSConv-v2 (Final) [TK] vẫn vượt qua 3Head-CBAM [TK] tới 4,47 điểm phần trăm mAP (70,73% so với 66,26%), cho thấy chất lượng kiến trúc đóng vai trò quyết định hơn kích thước tham số thuần túy.

Đối với tác động của Tucker Decomposition lên độ chính xác, ba cặp BF $\rightarrow$ TK cho thấy mức suy hao khác nhau: Paper-2H hầu như không mất mAP (60,11% $\rightarrow$ 60,47%, thực tế nhỉnh hơn do biến động thống kê trên tập nhỏ); 3Head-CBAM mất 2,18 điểm phần trăm (68,44% $\rightarrow$ 66,26%); GSConv-v2 mất 3,51% (74,24% $\rightarrow$ 70,73%). Mức suy hao tuyệt đối của GSConv-v2 là lớn nhất song cần đặt trong bối cảnh: mô hình khởi đầu từ nền mAP cao hơn và vẫn duy trì vị trí dẫn đầu sau nén. Nói cách khác, Tucker Decomposition với các hệ số nén được thiết lập trong đề tài kiểm soát được sự đánh đổi giữa nén mô hình và độ chính xác ở mức chấp nhận được cho ứng dụng thực tế.

Do chỉ test trên 108 hình ảnh nên $mAP@0.5$ sẽ không bằng so với test trên tập dữ liệu gốc. Tuy nhiên, ta vẫn có thể đánh giá được các thông số về inference time và chỉ số

[mAP@0.5](#). Cho thấy mô hình GSConv-v2 (Final) có chỉ số mAP hơn 10% so với mô hình baseline.

5.4 NHẬN XÉT

Quá trình triển khai mô hình GSConv-v2 (Final) [TK] lên Raspberry Pi 4 Model B 8 GB RAM cho thấy pipeline cải tiến đề xuất hoàn toàn khả thi trên phần cứng nhúng phổ thông, không yêu cầu bộ tăng tốc phần cứng chuyên dụng (NPU, GPU hay Edge TPU). Với thời gian suy diễn trung bình 166,5 ms mỗi khung hình (6,0 FPS) đo trực tiếp trên thiết bị, mô hình đáp ứng được yêu cầu thiết kế đặt ra trong Mục 5.1.3: tốc độ suy diễn dưới 300 ms mỗi khung hình trên CPU đơn luồng ARM Cortex-A72, đồng thời vượt mục tiêu khoảng 1,8 lần.

Về mặt tích cực, ba điểm nổi bật đáng ghi nhận từ kết quả triển khai. Thứ nhất, mô hình đề xuất GSConv-v2 (Final) [TK] là nhanh nhất trong nhóm ba mô hình được kiểm thử trên RPi4, nhanh hơn baseline [TK] 19,2 ms và nhanh hơn 3Head-CBAM [TK] đến 50,6 ms dù có kiến trúc phức tạp hơn. Điều này chứng minh rằng C3GhostV2 và GSConvV2 không chỉ hiệu quả trên GPU mà còn mang lại lợi ích tốc độ thực sự trên CPU ARM, nơi bộ nhớ cache nhỏ và băng thông giới hạn hơn. Thứ hai, dung lượng mô hình sau lượng tử hóa INT8 chỉ 10,74 MB, nhỏ hơn nhiều so với phiên bản floating-point (28,50 MB), giúp mô hình nạp hoàn toàn vào bộ nhớ và không gây áp lực lên heap RAM ngay cả khi chạy đồng thời với pipeline OpenCV xử lý video. Thứ ba, tỉ lệ $mAP@0.5 = 70,73\%$ đo trên 108 ảnh kiểm thử tại thiết bị — tuy thấp hơn kết quả GPU do tập test nhỏ hơn — vẫn duy trì khoảng cách hơn 10 điểm phần trăm so với baseline [TK] (60,47%), xác nhận các cải tiến kiến trúc được bảo toàn qua nguyên vẹn quá trình lượng tử hóa INT8.

Về mặt hạn chế, tốc độ 6,0 FPS chưa đạt ngưỡng real-time thông thường (≥ 25 FPS), có nghĩa là hệ thống phù hợp nhất cho các kịch bản giám sát tốc độ thấp hoặc theo dõi theo sự kiện (event-driven), chưa phù hợp với bài toán phát hiện tức thời trong dòng giao thông di chuyển nhanh. Ngoài ra, XNNPACK Execution Provider — mặc dù là bộ tăng tốc mặc định của ONNX Runtime cho CPU ARM — chỉ tận dụng hết các phép tính nhân nguyên INT8 khi kernel kích thước và alignment phù hợp; một số lớp C3GhostV2 có thể chạy dưới dạng float32 fallback, làm giảm hiệu quả tăng tốc INT8 dự kiến. Cuối cùng, các thước đo hiệu năng trong đề tài chỉ bao gồm thời gian forward pass thuần túy, chưa tính thời gian tiền xử lý (letterbox, normalize) và hậu xử lý (NMS, vẽ kết quả) — trên RPi4 hai bước này

có thể chiếm thêm 30–60 ms, hạ FPS thực tế xuống khoảng 3,5–4,5 FPS trong pipeline đầy đủ.

Nhìn chung, kết quả triển khai xác nhận tính khả thi kỹ thuật của hệ thống phát hiện người và phương tiện thời gian thực chi phí thấp trên nền tảng Raspberry Pi 4. Pipeline cải tiến ba giai đoạn đề xuất đã thành công trong việc đưa một mô hình phát hiện đối tượng hiệu suất cao ($mAP > 80\%$ trên GPU) về kích thước và tốc độ phù hợp với thiết bị nhúng phổ thông, mà không đòi hỏi phần cứng tăng tốc chuyên biệt. Đây là cơ sở thực tiễn để phát triển các ứng dụng giám sát thông minh nhúng trong tương lai, đặc biệt khi kết hợp với các nền tảng có khả năng tốt hơn như Raspberry Pi 5 hay Jetson Nano.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 KẾT LUẬN

Đề tài đã nghiên cứu, thiết kế và triển khai thành công một pipeline cải tiến ba giai đoạn cho mô hình phát hiện đối tượng Tiny YOLOv3, hướng đến bài toán phát hiện người đi bộ và phương tiện trong điều kiện thực tế trên thiết bị nhúng Raspberry Pi 4. Kết quả đạt được vượt các mục tiêu đề ra ban đầu trên cả ba phương diện: độ chính xác, hiệu quả tính toán và khả năng triển khai thực tế.

Về độ chính xác, mô hình cuối (M6_Tucker) đạt $mAP@0.5 = 81,89\%$ trên tập validation 4.052 ảnh — tăng +22,41 điểm phần trăm so với baseline Tiny YOLOv3 gốc (59,48%). Trong đó, lớp person đạt $AP = 79,77\%$ (tăng 17,26) và lớp car đạt $AP = 84,01\%$ (tăng 27,57). Việc bổ sung đầu phát hiện tại stride 8 (52×52) là yếu tố đóng góp lớn nhất, giải quyết triệt để điểm yếu của Tiny YOLOv3 gốc với đối tượng nhỏ và xa; cơ chế chú ý CBAM củng cố thêm khả năng lọc đặc trưng quan trọng khỏi nhiễu nền.

Về hiệu quả tính toán, pipeline kết hợp GSConv-v2, C3Ghost-v2 và Tucker Decomposition đã giảm số tham số từ 8,68 triệu (baseline) xuống còn 2,78 triệu (−67,9%) và GFLOPs từ 2,72 xuống 1,23 (−54,8%), trong khi duy trì mAP trên 81,8%. Điều này được thực hiện mà không phải đánh đổi tốc độ suy diễn trên GPU GTX 1650, M6_Tucker đạt 180 FPS, cao hơn cả baseline gốc (162,1 FPS).

Về triển khai thực tế, mô hình GSConv-v2 (Final) [TK] sau lượng tử hóa INT8 đạt thời gian suy diễn 166,5 ms (6,0 FPS) trên Raspberry Pi 4 Model B 8 GB RAM, không cần bất kỳ bộ tăng tốc phần cứng chuyên biệt nào. Kết quả này đáp ứng ràng buộc triển khai nhúng đề ra và phù hợp với các kịch bản giám sát giao thông điểm cố định tốc độ thấp trong thực tế.

Nhìn tổng thể, đề tài đã chứng minh rằng việc kết hợp có hệ thống các kỹ thuật cải tiến kiến trúc (3-Head, CBAM, GSConv, C3Ghost) với kỹ thuật nén mô hình (Tucker Decomposition) và tối ưu hóa suy diễn (ONNX Runtime INT8) là một hướng tiếp cận khả thi và hiệu quả để đưa mô hình phát hiện đối tượng hiệu suất cao xuống chạy trên phần cứng nhúng thương mại phổ thông.

6.2 HẠN CHẾ

Mặc dù đạt được những kết quả tích cực, đề tài vẫn còn một số hạn chế cần nhận nhận thẳng thắn để định hướng cải tiến trong tương lai.

Thứ nhất, tốc độ suy diễn trên Raspberry Pi 4 đạt 6,0 FPS (forward pass thuần túy) chưa đáp ứng yêu cầu real-time đầy đủ. Khi tính thêm thời gian tiền xử lý và hậu xử lý NMS, FPS thực tế trong pipeline hoàn chỉnh ước tính giảm xuống còn 3,5–4,5 FPS, giới hạn phạm vi ứng dụng ở các kịch bản tốc độ thấp.

Thứ hai, đề tài chỉ đánh giá trên hai lớp đối tượng (person và car) với tập dữ liệu kết hợp Caltech Pedestrian và KITTI, vốn được thu thập chủ yếu trong điều kiện ban ngày và tầm nhìn rõ ràng. Hiệu năng mô hình trong điều kiện đêm tối, mưa sương, hay ánh sáng ngược chưa được kiểm chứng.

Thứ ba, phương pháp Tucker Decomposition được áp dụng thủ công với các hệ số rank ratio được chọn qua thực nghiệm, chưa có cơ chế tự động tìm kiếm hệ số nén tối ưu toàn cục (global rank selection). Điều này đòi hỏi kỹ sư phải lặp lại thực nghiệm nhiều lần và có thể bỏ qua các cấu hình nén tốt hơn.

Thứ tư, quy trình lượng tử hóa INT8 sử dụng Post-Training Quantization với tập hiệu chỉnh (calibration set) nhỏ, có thể gây suy hao độ chính xác cao hơn so với Quantization-Aware Training (QAT) — đặc biệt với các lớp tích chập đầu và các lớp CBAM nhạy cảm với phân phối activation.

Cuối cùng, đề tài chưa thực hiện đánh giá trong điều kiện phần cứng thực tế đầy đủ, bao gồm độ ổn định nhiệt (thermal throttling) khi Pi chạy liên tục nhiều giờ đồng hồ — một yếu tố quan trọng trong triển khai camera giám sát dài hạn.

6.3 HƯỚNG PHÁT TRIỂN

Dựa trên các kết quả đạt được và hạn chế còn tồn tại, đề tài đề xuất các hướng nghiên cứu và phát triển tiếp theo như sau.

Hướng thứ nhất là nâng cấp lên phần cứng mạnh hơn. Raspberry Pi 5 với CPU ARM Cortex-A76 hoạt động ở 2,4 GHz có thể cải thiện tốc độ suy diễn ước tính 2–3× so với Pi 4, đưa FPS thực tế lên ngưỡng 10–15 FPS. Các nền tảng có NPU tích hợp như NVIDIA Jetson Nano (128-core GPU) hay Coral Dev Board (Edge TPU) cho phép đạt real-

time thực sự (≥ 25 FPS) với cùng mô hình, mở rộng phạm vi ứng dụng sang giám sát giao thông tốc độ cao.

Hướng thứ hai là tự động hóa tìm kiếm hệ số nén (Automated Rank Selection). Thay vì chọn rank ratio thủ công, có thể tích hợp thuật toán tìm kiếm dựa trên sensitivity analysis đo mức suy hao mAP khi nén từng lớp độc lập — hoặc sử dụng Neural Architecture Search (NAS) để đồng thời tối ưu kiến trúc và tỉ lệ nén theo ràng buộc FLOPs mục tiêu.

Hướng thứ ba là áp dụng Quantization-Aware Training (QAT) thay thế Post-Training Quantization. QAT mô phỏng lỗi lượng tử hóa trong quá trình huấn luyện, giúp mô hình tự thích nghi với giới hạn INT8 và thường giảm suy hao mAP từ 1–3 xuống còn dưới 0,5 đặc biệt quan trọng với các lớp CBAM và detection head nhạy cảm.

Hướng thứ tư là mở rộng tập dữ liệu và lớp đối tượng. Bổ sung dữ liệu thu thập trong điều kiện đêm (IR camera), thời tiết xấu (mưa, sương) và góc camera đa dạng sẽ cải thiện độ bền (robustness) của mô hình. Mở rộng sang thêm lớp đối tượng như xe máy, xe đạp, người đi bộ mặc mưa áo hay các đối tượng đặc thù giao thông Việt Nam sẽ tăng tính ứng dụng thực tế.

Hướng thứ năm là tích hợp vào hệ thống giám sát thực tế. Kết hợp mô hình phát hiện đối tượng với thuật toán tracking (SORT, ByteTrack) để theo dõi quỹ đạo di chuyển, đếm phương tiện và cảnh báo vi phạm giao thông. Phát triển giao diện người dùng trên Raspberry Pi với camera module và hệ thống cảnh báo (buzzer, LED, gửi thông báo qua MQTT) để hoàn thiện sản phẩm ứng dụng cuối.

TÀI LIỆU THAM KHẢO

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788. <https://doi.org/10.1109/CVPR.2016.91>
- [2] Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767. <https://arxiv.org/abs/1804.02767>
- [3] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. Advances in Neural Information Processing Systems (NeurIPS), 28, 91–99.
- [4] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117–2125.
- [5] Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2010). The PASCAL Visual Object Classes (VOC) challenge. International Journal of Computer Vision, 88(2), 303–338. <https://doi.org/10.1007/s11263-009-0275-4>
- [6] Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., & Ren, D. (2020). Distance-IoU loss: Faster and better learning for bounding box regression. Proceedings of the AAAI Conference on Artificial Intelligence, 34(7), 12993–13000. <https://doi.org/10.1609/aaai.v34i07.6999>
- [7] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. Proceedings of the 32nd International Conference on Machine Learning (ICML), 448–456.

- [8] Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional block attention module. *Proceedings of the European Conference on Computer Vision (ECCV)*, 3–19. https://doi.org/10.1007/978-3-030-01234-2_1
- [9] Li, H., Li, J., Wei, H., Liu, Z., Zhan, Z., & Ren, Q. (2022). Slim-neck by GSConv: A better design paradigm of detector architectures for autonomous driving. *arXiv preprint arXiv:2206.02424*. <https://arxiv.org/abs/2206.02424>
- [10] Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., & Xu, C. (2020). GhostNet: More features from cheap operations. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 1577–1586.
- [11] Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7464–7475.
- [12] Tucker, L. R. (1966). Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3), 279–311. <https://doi.org/10.1007/BF02289464>
- [13] De Lathauwer, L., De Moor, B., & Vandewalle, J. (2000). A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21(4), 1253–1278. <https://doi.org/10.1137/S0895479896305696>
- [14] Kim, Y.-D., Park, E., Yoo, S., Choi, T., Yang, L., & Shin, D. (2016). Compression of deep convolutional neural networks for fast and low power mobile applications. *Proceedings of the International Conference on Learning Representations (ICLR 2016)*. <https://arxiv.org/abs/1511.06530>

- [15] Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for autonomous driving? The KITTI vision benchmark suite. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3354–3361.
- [16] Dollár, P., Wojek, C., Schiele, B., & Perona, P. (2012). Pedestrian detection: An evaluation of the state of the art. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(4), 743–761. <https://doi.org/10.1109/TPAMI.2011.155>
- [17] Falaschetti, L., Manoni, L., Palma, L., Pierleoni, P., & Turchetti, C. (2024). Embedded real-time vehicle and pedestrian detection using a compressed Tiny YOLO v3 architecture. *IEEE Transactions on Intelligent Transportation Systems*, 25(12), 19399–19414. <https://doi.org/10.1109/TITS.2024.3447453>